

Healthcare Fraud Detection using Machine Learning

Healthcare Fraud is one of the major problems in the US Healthcare System, and it leads to huge financial losses and also impacts the quality of healthcare facilities. The Payers or Insurance Agencies like Medicare, Medicaid, and other agencies have to spend more money, and to incur the financial losses, they have to decide to increase the insurance premium amount. Eventually, the healthcare system is becoming more and more expensive.

Not only is the financial loss a major concern, but fraud also severely hinders the quality of healthcare being provided to legitimate patients. There are ways of detecting fraud manually but it requires intensive medical research and is also too time-consuming.

Hence, effective fraud detection using sophisticated machine learning techniques along with aided medical domain expertise is very important for improving the quality and reducing the cost of healthcare services.

Table of Contents

1. Prerequisites
2. Business Problem
3. Machine Learning Formulation of the Business Problem
4. Business Constraints
5. Source of Data
6. Performance Metrics
7. Existing Approaches and Improvements over them
8. Exploratory Data Analysis (EDA)
9. Data Pre-processing
10. Machine Learning Models — Hyperparameter Tuning and Model Training
11. Comparison of Models
12. Model Interpretability
13. Deployment
14. Future Work
15. References
16. GitHub Repository

Prerequisites

1. Understanding of Statistical Concepts/Plots.
2. Concepts of Machine Learning.

Business Problem

Fraud, Waste, and Abuse have become one of the biggest concerns in the healthcare system of the United States and can be caused by different healthcare entities (**3 Ps**) viz, **Patients** (Insurance Subscriber/Beneficiary), Healthcare **Providers** (Hospitals, Physicians, Laboratories, Pharmacies, etc) and **Payers** (Insurance Providers like Medicare, Medicaid, and other agencies). Frauds can be caused by either a single entity or by a conspiracy of multiple entities. In this case study, we primarily focus on the frauds caused by the Providers as part of insurance claims submitted to the Insurance Agencies.

Many healthcare systems rely on domain experts to manually review insurance claims and detect potential frauds. There are hundreds of thousands to millions of claims data getting generated over time. Manual review of the claims is very time-consuming and challenging. To reduce such effort, various machine learning techniques can be leveraged to automate the manual processing of the claims and identify the potential frauds. Such a system would certainly help in reducing the cost, manual effort, processing & review time, and other challenges in the system. This case study aims to solve the problem of detecting potential frauds in a healthcare system and assist in making the healthcare system better.

Machine Learning Formulation of the Business Problem

Identify the potential frauds made by the Providers, in the insurance claims submitted to the Insurance Agencies. The problem can be projected into a **binary classification task** of determining whether the claim record(s) submitted by the Providers is/are fraudulent or legitimate. Given the data related to the Providers and the claim submitted by them, a Machine Learning Model can be built to determine whether a claim record is fraudulent or not.

Business Constraints

1. **No Latency Requirement:** As the current process of manually labeling the claims as fraudulent or not, takes much time, there won't be a strict need to have a low-latency system for fraud detection. However, the time taken should be reasonable depending on the business need.
2. **Interpretability:** Interpretability is very important as the machine learning model needs to justify why it classifies a claim as fraudulent or not. There could be legal proceedings involved on the

Providers after identifying a fraudulent claim filed by them. Hence, it is of utmost importance that the Machine Learning Model gives proper justification in case of detecting fraud.

If a legitimate Provider is detected as fraud and has to undergo legal proceedings, it may plan to restrict its facility and cooperation with the Government, and the reachability of the healthcare systems will be compromised.

3. **Errors:** False Negative Rate (FNR) should be as low as possible. In some healthcare systems which conduct some manual review after the machine classification, a slightly higher False Positive Rate (FPR) is acceptable. However, FNR should always be minimum, otherwise, the actual fraud Providers would escape and continue to impact the healthcare system.
4. **Class Probabilities:** It would be better to provide class probabilities so that the user can know how probable a particular fraudulent claim is.

Source of Data

The data is taken from Kaggle:

HEALTHCARE PROVIDER FRAUD DETECTION ANALYSIS
HEALTHCARE PROVIDER FRAUD DETECTION ANALYSIS
HEALTHCARE PROVIDER FRAUD DETECTION ANALYSISwww.kaggle.com

Dataset Details

The dataset is present in the form of excel files and there are a total of eight files, four for each train and test data.

These are:

1. **Mapping Data** (Train-1542865627584.csv and Test-1542969243754.csv): Contains the mapping of Provider's unique id and the class label signifying whether the Provider is fraud or not. For the Test data, only the Provider's unique id is given and the class label has to be found.
2. **Beneficiary Data** (Train_Beneficiarydata-1542865627584.csv and Test_Beneficiarydata-1542969243754.csv): Contains the demographic, disease, premium, and reimbursement related information of the beneficiaries/patients.
3. **Inpatient Data** (Train_Inpatientdata-1542865627584.csv and Test_Inpatientdata-1542969243754.csv): Provides the details of the claims filed for the patients who were admitted to the hospitals for their treatment.

4. **Outpatient Data** (Train_Outpatientdata-1542865627584.csv and Test_Outpatientdata-1542969243754.csv): Provides the details related to the claims filed for the patients who visited the hospitals but were not admitted.

Column Details

Each of the columns present in the excel sheets is described below:

- **Mapping Data**

Mapping Data is present for both training and testing.

Train Data:

1. **Provider:** Unique identifier for a Provider (Hospital, Pharmacy, Laboratories, etc).
2. **Potential Fraud:** Class label to indicate whether the Provider is a fraud or not.

Test Data: Test data only contains the unique identifier for a Provider and not a class label as the class label has to be found out.

- **Beneficiary Data**

1. **BeneID:** Unique identifier of the patient/beneficiary who is registered to the health insurance provided by the Payer (Insurance Agencies like Medicare, Medicaid, and other agencies).
2. **DOB:** Date of birth of the beneficiary.
3. **DOD:** Date of death of the beneficiary in case the beneficiary is dead. Otherwise, this contains the NaN value.
4. **Gender, Race, State, Country:** Contains numeric code for gender, race, state, and country of the beneficiary.
5. **RenalDiseaseIndicator:** Flag to indicate whether the beneficiary has any problem related to kidney failure or not.
6. **Chronic Condition columns:** Indicate whether a beneficiary has a disease for more than a year and requires ongoing medical attention or limited activities of daily living or both. There are columns to indicate whether the beneficiary has chronic conditions related to Alzheimer's, heart failure, kidney disease, cancer, obstructive pulmonary, depression, diabetes, ischemic heart disease, osteoporosis, rheumatoid arthritis, and stroke.
7. **IPAnnualReimbursementAmt:** annual amount reimbursed for the treatment of the beneficiary when admitted to the hospital.
8. **IPAnnualDeductibleAmt:** annual premium amount paid to the Insurance Agency towards the

treatment of the beneficiary when he was admitted to the hospital.

9. **OPAnnualReimbursementAmt**: annual amount reimbursed for the treatment of the beneficiary when visited the hospital but not admitted.

10. **OPAnnualDeductibleAmt**: annual premium amount paid to the Insurance Agency towards the treatment of the beneficiary when he visited the hospital but was not admitted.

- **Inpatient and Outpatient Data:**

1. **BeneID**: Unique identifier of the beneficiary.

2. **ClaimID**: Unique identifier of the health insurance claim.

3. **ClaimStartDt** and **ClaimEndDt**: Dates indicating when the insurance claim was submitted and when it was settled and closed.

4. **Provider**: Unique identifier for a Provider.

5. **InscClaimAmtReimbursed**: Amount reimbursed by the Payer (Insurance Agency) for the healthcare services provided to the beneficiary.

6. **Physicians-related columns**: Columns showing the physicians who attended the beneficiary/patient, operated the patient, and any other physicians if any.

7. **Admission and Discharge Date (Inpatient Data)**: Columns showing the dates on which the beneficiary was admitted to the hospital and when he was discharged.

8. **ClmAdmitDiagnosisCode**: Diagnosis code indicating the beneficiary's initial diagnosis at admission.

9. **Claim Diagnosis Codes 1–10:**

ClmDiagnosisCode_1: Diagnosis code identifying the beneficiary's principal diagnosis.

ClmDiagnosisCode_2–10: Diagnosis code in the 2nd, 3rd, and so on, till the 10th position identifying the condition(s) for which the beneficiary is receiving care.

10. **Claim Procedure Codes 1–6**: Codes that indicate the principal or other procedures performed during the period covered by the institutional claim.

11. **DiagnosisGroupCode (Inpatient Data)**: Code to classify hospital cases according to certain groups, also referred to as DRGs, which are expected to have similar hospital resource use (cost).

12. **DeductibleAmtPaid**: Amount the beneficiary has to pay as part of the claim, and the rest of the amount is paid by the insurance company. It is equal to the total claim amount minus the reimbursed amount.

Performance Metrics

To evaluate how well a Machine Learning Model is built, the below set of Performance Metrics is used.

I. Binary Log Loss: Important metric when the ML Model gives the probability estimates. It lies in the range from zero to infinity. The lower the value, the better is the ML Model.

Given by the equation:

$$-\frac{1}{N} \left\{ \sum_{i=1}^n y_i \cdot \log(P(\hat{y}_i)) + (1 - y_i) \cdot \log(1 - P(\hat{y}_i)) \right\}$$

where,

N: Total number of datapoints.

y_i : Actual class label and has a value either 0 or 1.

$P(\hat{y}_i)$: Predicted probability estimates.

Binary Log-loss

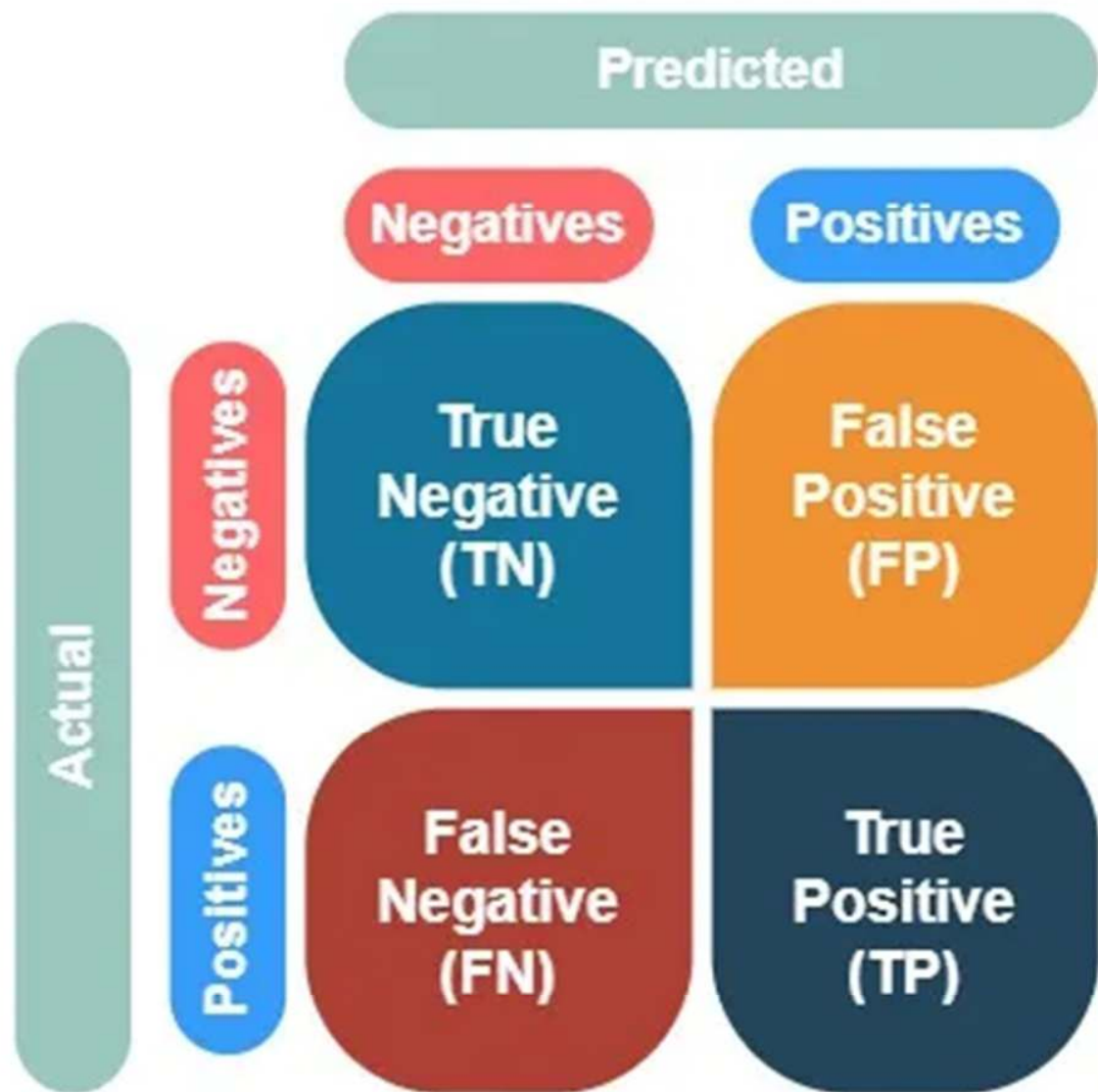
```
1  def getLogloss(yActual, yPredProb, datasetType='Test'):
2      """
3      Finds and displays the Log Loss
4
5      Parameters:
6      -----
7      yActual: array-like
8          Ground truth (correct) class labels for 'n' samples.
9      yPredProb: array-like
10         Predicted probabilities, as returned by a model's predict_proba method.
11     datasetType: str
12         Type of Dataset: Test or Train.
13     """
14
15     logloss = log_loss(y_true=yActual, y_pred=yPredProb)
16
17     print('Log-loss of the Model on ', datasetType, ' Dataset: ', logloss)
```

II. Performance Metric: Confusion Matrix and other related metrics:

We will use Confusion Matrix and derive different metrics from the values in the Confusion Matrix. Confusion Matrix is a good metric for binary classification and also robust to class imbalance.

1. Confusion Matrix:

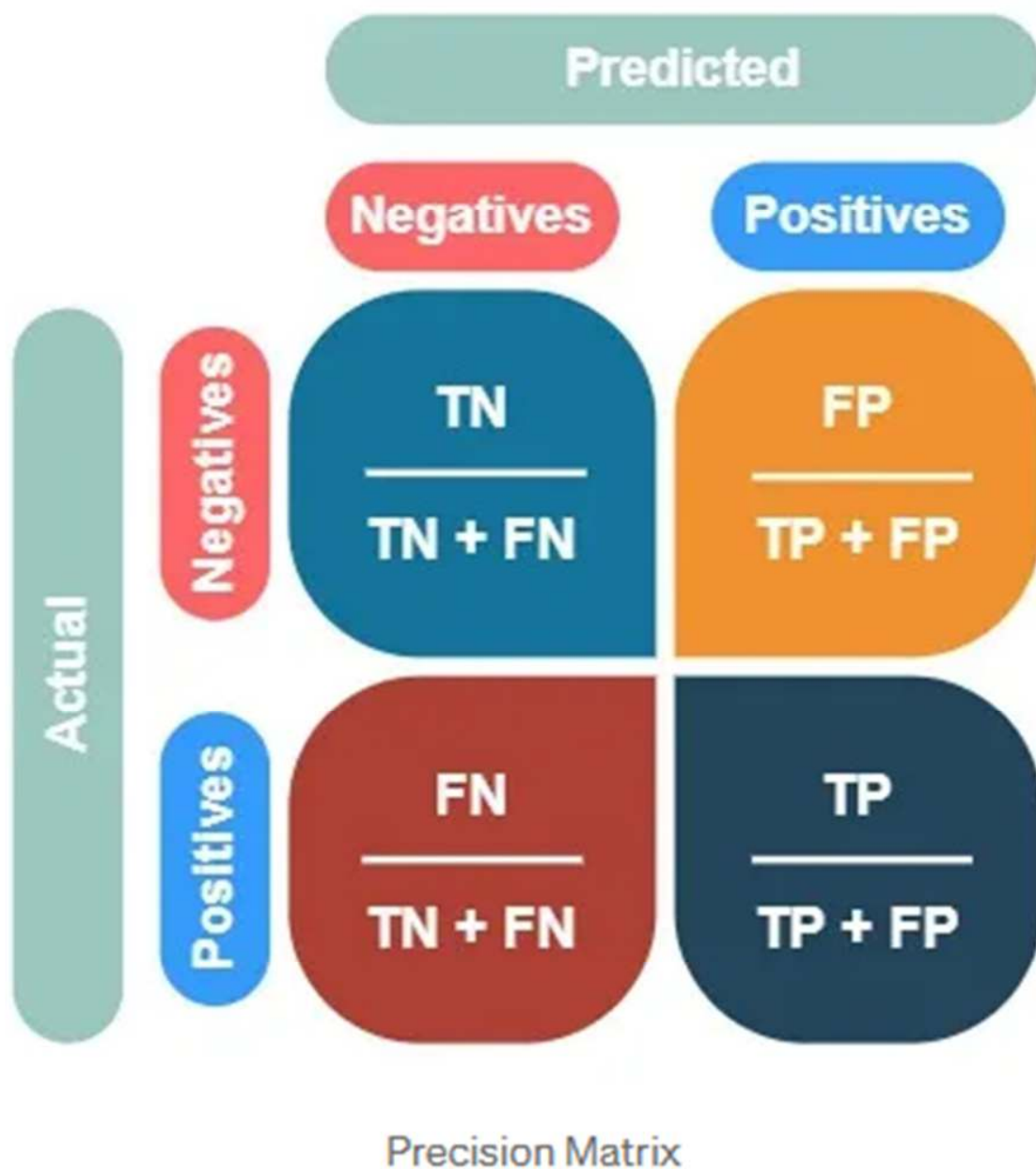
A confusion matrix is a way of assessing the performance of a classification model. It is a comparison between the ground truth (actual values) and the predicted values emitted by the model for the target variable.



Confusion Matrix

3. Precision Matrix:

Precision Matrix helps in knowing the Precision values which lie on the diagonal of the matrix.



Precision or **Positive Predicted Value (PPV)** indicates “Of all predicted positives, how many are actual positives”.

Given by the equation:

Press enter or click to view image in full size

$$\text{Precision (Positive Predictive Value (PPV))} = \frac{TP}{TP + FP}$$

where,

TP (True Positive): an outcome where the model correctly predicts the positive class .

FP (False Positive): an outcome where the model incorrectly predicts the positive class.

Precision or Positive Predicted Value (PPV)

The other diagonal in the Precision Matrix tells “Of all predicted negatives, how many are actual negatives”.

Given by the equation:

Press enter or click to view image in full size

$$\text{Negative Predictive Value (NPV)} = \frac{TN}{TN + FN}$$

where,

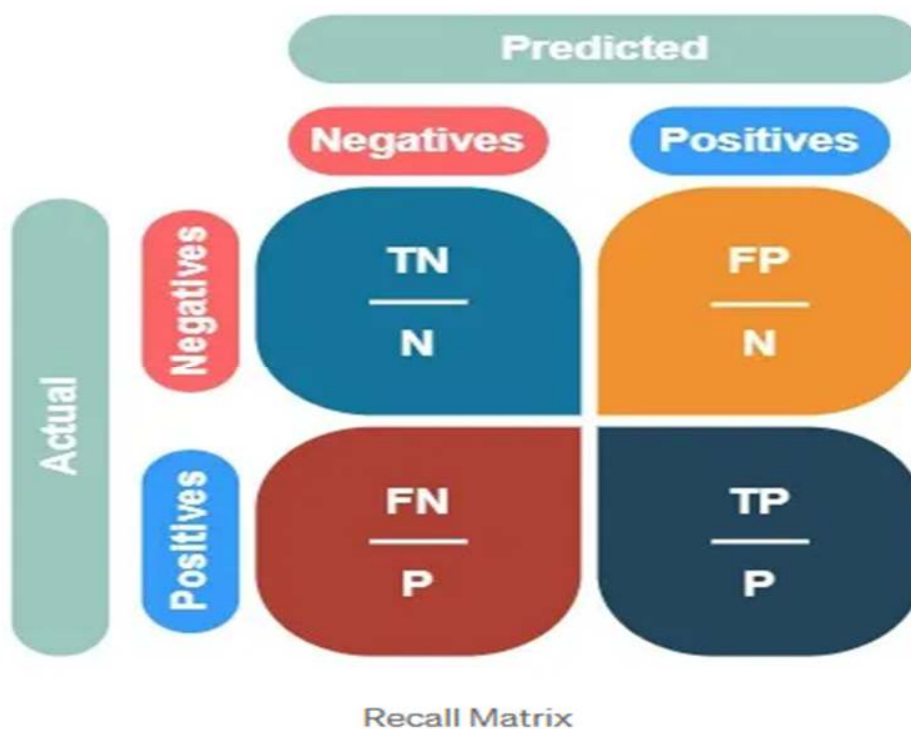
TN (True Negative): an outcome where the model correctly predicts the negative class.

FN (False Negative): an outcome where the model incorrectly predicts the negative class.

Negative Predicted Value (NPV)

4. Recall Matrix:

Recall Matrix helps in knowing the Recall (or Sensitivity) and Specificity values, which lie on the diagonal elements.



Recall/Sensitivity or True Positive Rate (TPR) indicates “Of all actual positives, how many are predicted positives”.

Given by the equation:

$$\text{Recall/Sensitivity} = \frac{TP}{P}$$

Recall/Sensitivity or True Positive Rate

Specificity indicates “Of all actual negatives, how many are predicted negatives”.

Given by the equation:

$$\text{Specificity} = \frac{TN}{N}$$

Specificity or True Negative Rate

4. False Positive Rate (FPR) and False Negative Rate (FNR):

Fall-out or False Positive Rate (FPR) indicates “Of all actual negatives, how many are predicted positives”.

Given by the equation:

$$\text{FPR} = \frac{FP}{N}$$

Fall-out or False Positive Rate

Miss rate or False Negative Rate (FNR) indicates “Of all actual positives, how many are predicted negatives”.

Given by the equation:

$$\text{FNR} = \frac{FN}{P}$$

Miss rate or False Negative Rate

III. Balanced Accuracy (BACC): Focuses on high recall (TPR and TNR).

Computes balanced accuracy and is the average of recall obtained on each class.

Given by the equation:

Press enter or click to view image in full size

$$\text{BACC Score} = \frac{\text{Recall of +ve class} + \text{Recall of -ve class}}{2}$$

Balanced Accuracy

IV. Matthew’s Correlation Coefficient (MCC): Considers all the values from a Confusion Matrix and gives more value when the predictions are correct.

Given by the equation:

Press enter or click to view image in full size

$$\text{MCC Score} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Matthew’s Correlation Coefficient

`def plotPerformanceMatrix(yActual, yPred):`

'''

Function to compute Confusion, Precision and Recall Matrix and plot them.

Parameters:

```

yActual: array-like
    Ground truth (correct) class labels for 'n' samples.
yPred: array-like
    Predicted class labels for 'n' samples.
'''

# Get Confusion Matrix based on the input 'yActual' and 'yPred'.
confusionMatrix = confusion_matrix(y_true=yActual, y_pred=yPred)

# Compute the Precision Matrix
precisionMatrix = (confusionMatrix/confusionMatrix.sum(axis=0))
# Divide each element of the confusion matrix with the sum of the elements in that column (total predicted
value)

# Compute the Recall Matrix
recallMatrix = (confusionMatrix.T/confusionMatrix.sum(axis=1)).T
# Divide each element of the confusion matrix with the sum of the elements in that row (total actual values)

plt.figure(figsize=(20,5))

# Plot the Confusion Matrix.
plt.subplot(131)
sns.heatmap(confusionMatrix, annot=True, fmt='d', cmap='Reds')
plt.title('Confusion Matrix on Test Data', fontsize=20)
plt.xlabel('Predicted Value', fontsize=15) # Label on the x-axis
plt.ylabel('Actual Values', fontsize=15) # Label on the y-axis
plt.xticks(ticks=[0.5, 1.5], labels=['Predicted: NO (Non-fraud)', 'Predicted: YES (Fraud)'])
plt.yticks(ticks=[0.5, 1.5], labels=['Actual: NO (Non-fraud)', 'Actual: YES (Fraud)'], rotation=0)

# Plot the Precision Matrix.
plt.subplot(132)
sns.heatmap(precisionMatrix, annot=True, fmt='.3f', cmap=sns.light_palette('green'))
plt.title('Precision Matrix on Test Data', fontsize=20)
plt.xlabel('Predicted Value', fontsize=15) # Label on the x-axis
plt.xticks(ticks=[0.5, 1.5], labels=['Predicted: NO (Non-fraud)', 'Predicted: YES (Fraud)'])

# Plot the Recall Matrix.
plt.subplot(133)
sns.heatmap(recallMatrix, annot=True, fmt='.3f', cmap='Blues')
plt.title('Recall Matrix on Test Data', fontsize=20)
plt.xlabel('Predicted Value', fontsize=15) # Label on the x-axis
plt.xticks(ticks=[0.5, 1.5], labels=['Predicted: NO (Non-fraud)', 'Predicted: YES (Fraud)'])
plt.show()

totalDatapoints = len(yActual) # Total number of Datapoints
print('Percentage of misclassified points (Test Data): ', ((totalDatapoints -
np.trace(confusionMatrix))/totalDatapoints)*100, '%')
print('Sum of columns in the Precision Matrix: ', precisionMatrix.sum(axis=0))

```

```

print('Sum of rows in the Recall Matrix: ', recallMatrix.sum(axis=1))

# Find Total Negatives and Total Positives (Actual/ ground truth)
totalNegative, totalPositive = tuple(confusionMatrix.sum(axis=1))

# Find True Positive, False Positive, False Negative and True Positive
tn, fp, fn, tp = confusionMatrix.ravel()

# Calculate the FPR and FNR
fpr = fp / totalNegative
fnr = fn / totalPositive
print('False Positive Rate (FPR) on Test Data: ', fpr)
print('False Negative Rate (FNR) on Test Data: ', fnr)

# Get the BACC Score
bacc = balanced_accuracy_score(y_true=yActual, y_pred=yPred)
print('Balanced Accuracy Score (BACC) on Test Data: ', bacc)

# Get the MCC Score
mcc = matthews_corrcoef(y_true=yActual, y_pred=yPred)
print('Matthew\'s Correlation Coefficient (MCC) on Test Data: ', mcc)

```

IV. F1-Score: F1-Score is a measure of a test's accuracy.

F1-Score considers both Precision and Recall and is the harmonic mean of them.

Given by the equation:

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1-Score

```

def getF1Score(yActual, yPred):
    """
    Calculates and displays the F1-Score

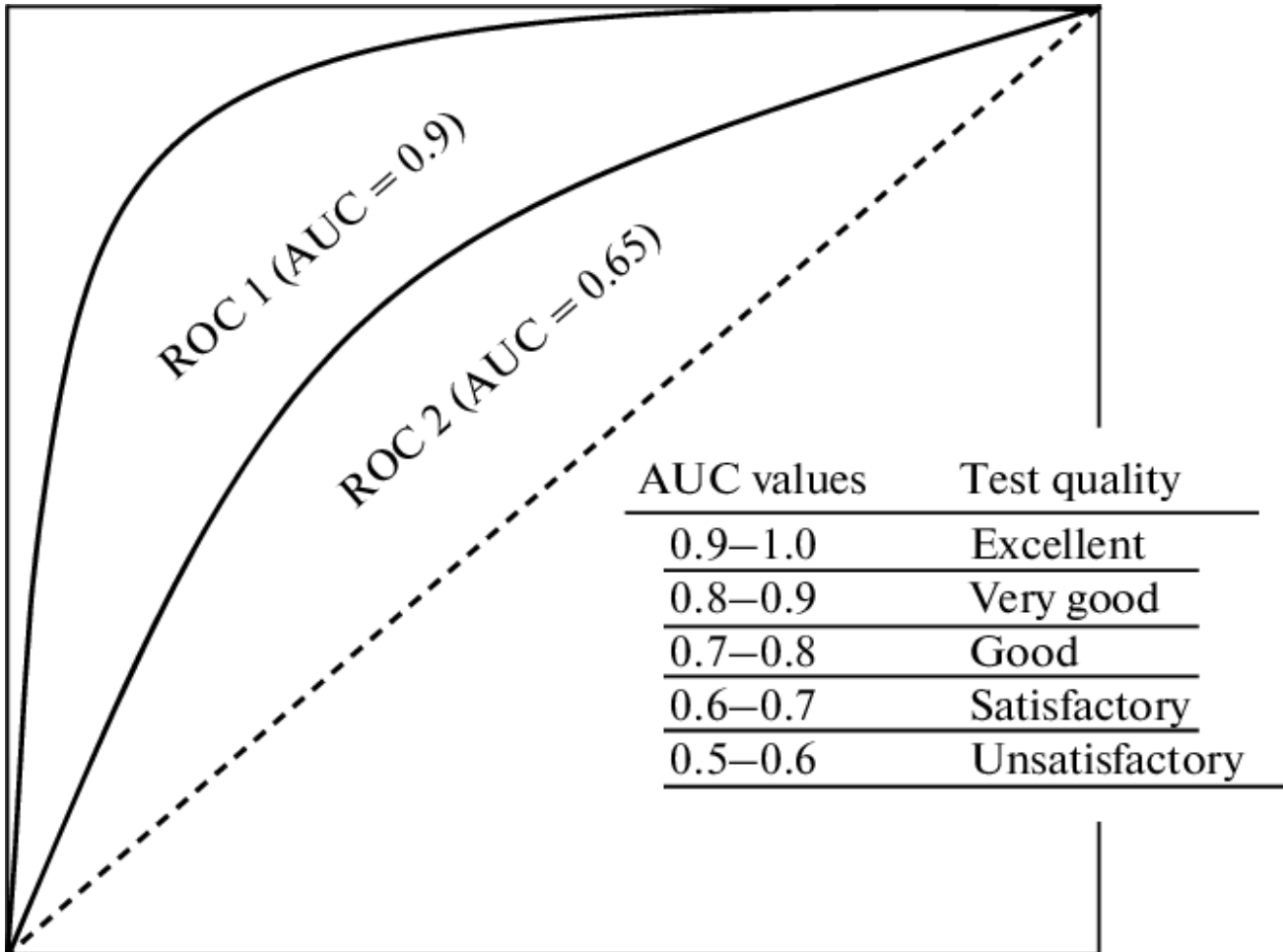
    Parameters:
    -----
    yActual: array-like
        Ground truth (correct) class labels for 'n' samples.
    yPred: array-like
        Predicted class labels for 'n' samples.
    """

    f1Score = f1_score(y_true=yActual, y_pred=yPred)

```

```
print('F1-Score of the Model on Test Data: ', f1Score)
```

V. Area Under the Curve (AUC): AUC is the area under the curve of Receiver Operating Characteristic (ROC).



```
def plotROC(yTestActual, yTestPredProb, yTrainActual=None, yTrainPredProb=None):  
    """  
    Plots the ROC and calculates the AUC  
  
    Parameters:  
    -----  
    yTestActual: array-like  
        Ground truth (correct) class labels for 'n' samples on Test Dataset.  
    yTestPredProb: array-like  
        Predicted probabilities, as returned by a model's predict_proba method on Test Dataset.  
    yTrainActual: array-like  
        Ground truth (correct) class labels for 'n' samples on Train Dataset.  
    yTrainPredProb: array-like  
        Predicted probabilities, as returned by a model's predict_proba method on Train Dataset.
```

'''

```
# Check if the input Predicted variable is a Probabilty Estimates or not
if len(yTestPredProb.shape) == 1:
    print('Please pass the correct Probability Estimates')
    return

# Calculate the Predicted Probability Estimates for the positive class of Test Data
yTestPredPE = getProbEstimates(yTestPredProb)

# Find the FPR, TPR and Threshold values for the Test Data
fprTest, tprTest, thresholdTest = roc_curve(yTestActual, yTestPredPE)

# Find the AUC
areaTest = auc(fprTest, tprTest)

# If Train Dataset to also to be considered for plotting ROC
if (type(yTrainActual) != type(None) and type(yTrainPredProb) != type(None)):

    # Check if the input Predicted variable is a Probabilty Estimates or not
    if len(yTrainPredProb.shape) == 1:
        print('Please pass the correct Probability Estimates')
        return

    # Calculate the Predicted Probability Estimates for the positive class of Train Data
    yTrainPredPE = getProbEstimates(yTrainPredProb)

    # Find the FPR, TPR and Threshold values for the Train Data
    fprTrain, tprTrain, thresholdTrain = roc_curve(yTrainActual, yTrainPredPE)

    # Find the AUC
    areaTrain = auc(fprTrain, tprTrain)

# Plot the ROC Curve
# If Train Dataset to also to be considered for plotting ROC
if (type(yTrainActual) != type(None) and type(yTrainPredProb) != type(None)):

    plt.plot(fprTrain, tprTrain, label= 'Train AUC: ' + str(areaTrain))

plt.plot(fprTest, tprTest, label= 'Test AUC: ' + str(areaTest))
plt.legend()
# If Train Dataset to also to be considered for plotting ROC
if (type(yTrainActual) != type(None) and type(yTrainPredProb) != type(None)):

    plt.title('ROC Curve of Train and Test Data', fontsize=20)

else:
```

```
plt.title('ROC Curve of Test Data', fontsize=20)

plt.xlabel('FPRs', fontsize=15)
plt.ylabel('TPRs', fontsize=15)
plt.grid()
plt.show()
```

Existing Approaches and Improvements over them

There are some existing solutions and research work on the problem and have been given in the references. The existing solutions use various oversampling and undersampling techniques but with limited feature engineering. In this solution, we would see that we have created many new features by doing feature engineering on various features. Have ensured that there is no data leakage when doing hyperparameter tuning with k-fold cross-validation using Random Search CV and, also during model training and evaluation. Also, implemented the Model Interpretability to explain why the model predicts a particular claim record as fraudulent or not.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) helps in analyzing the data using simple tools from statistics, plots, linear algebra, and others.

In EDA, we plot different graphs for getting a better understanding of the data. Let's start doing the univariate analysis of all the features using various plots and conclude each.

1. **EDA of Provider Dataset:**

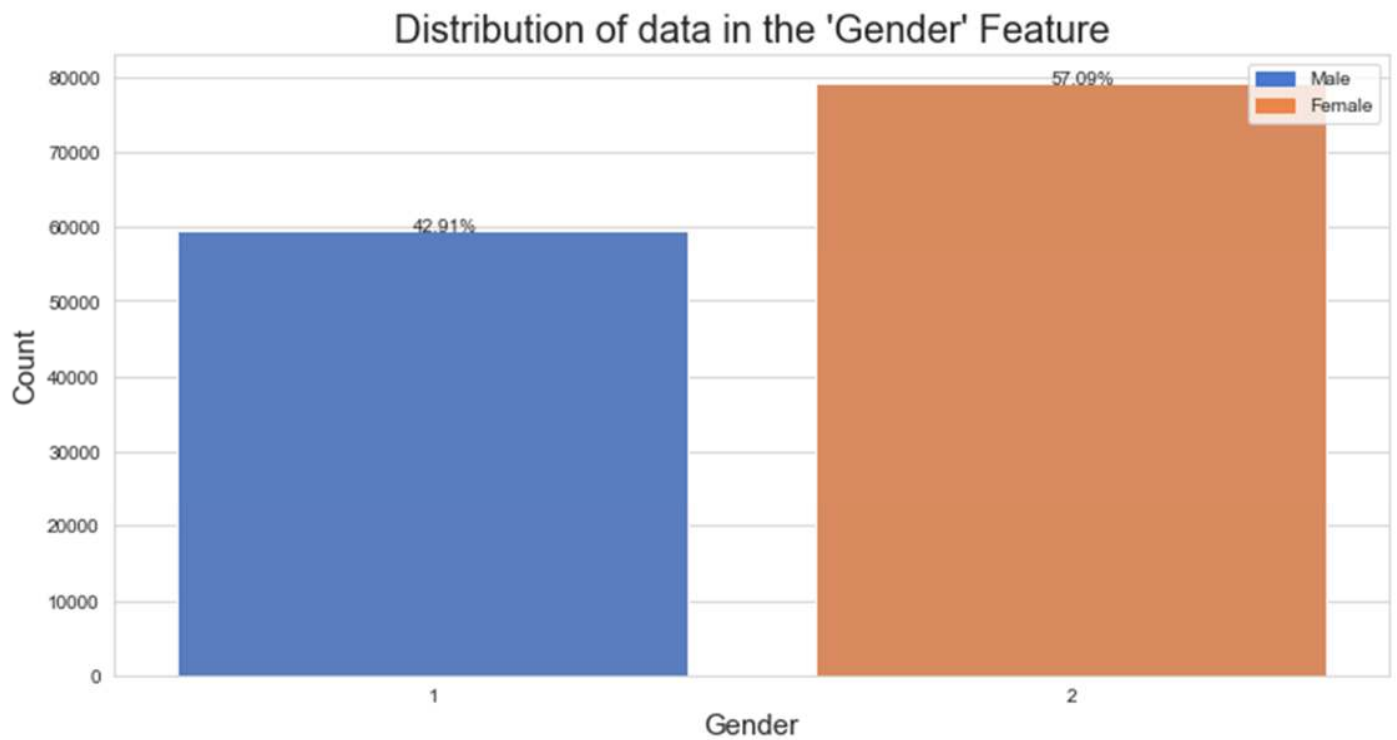


Observation:

- The dataset is **highly imbalanced** with 90.6% non-fraud Providers and 9.4% fraud Providers class labels.

2. EDA of Beneficiary Dataset:

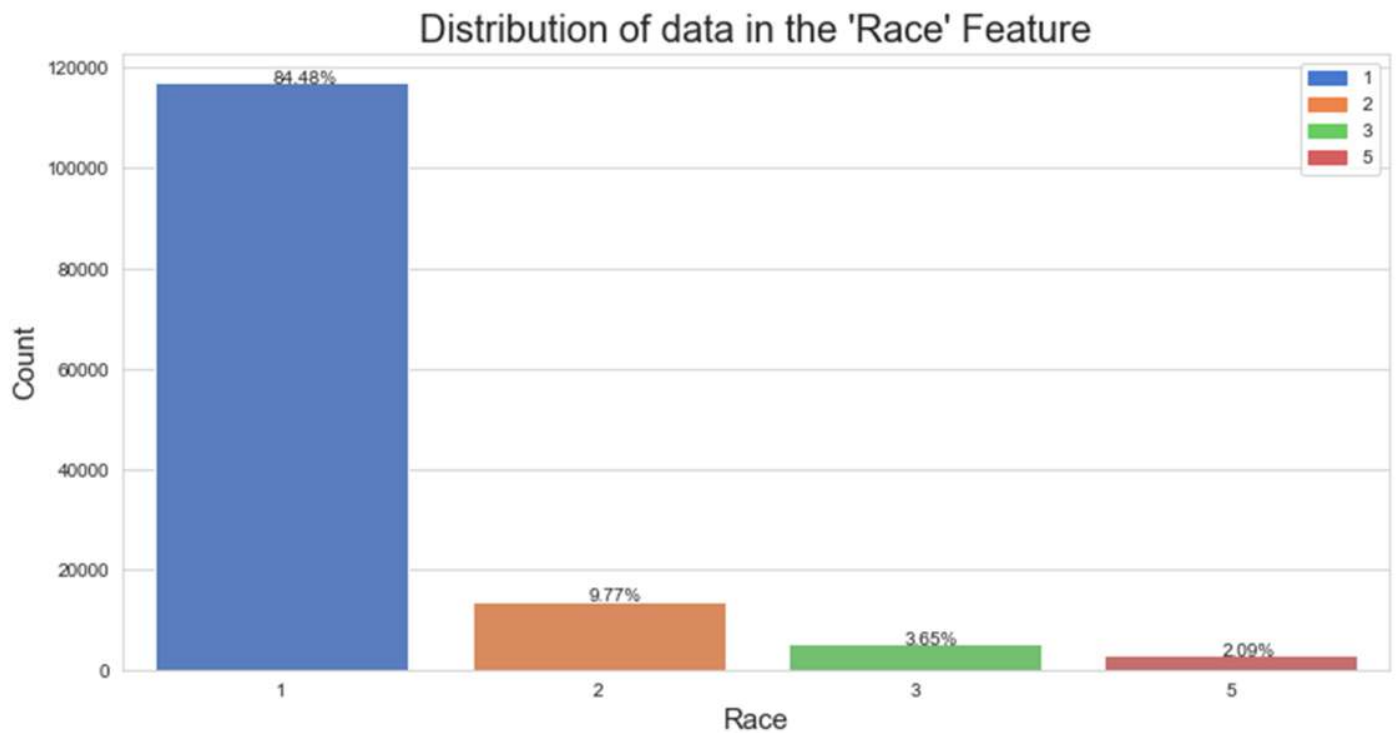
2.1. EDA of Beneficiary Dataset — Gender Feature:



Observation:

- The gender feature is almost balanced.
- 42.9% of the beneficiaries (patients) are males and the rest 57.1% are females.

2.2. EDA of Beneficiary Dataset — Race Feature:



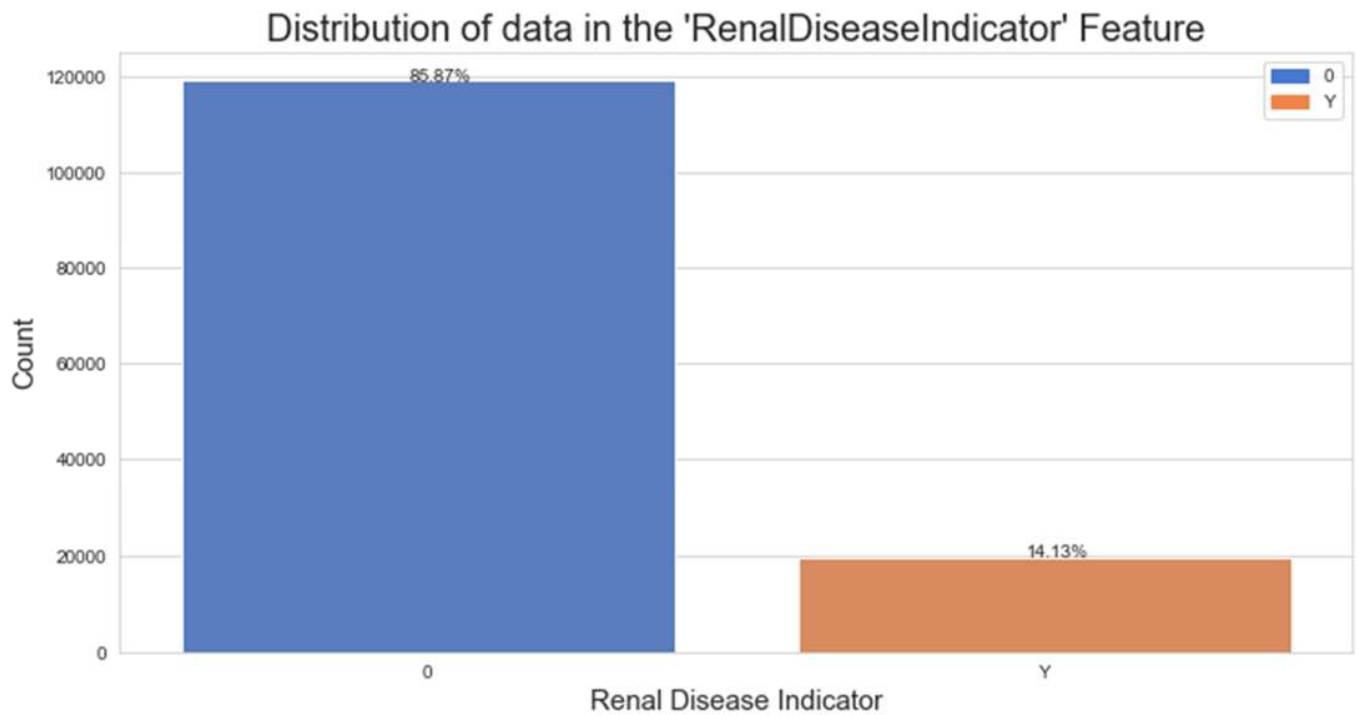
Observation:

- There are four different races encoded as: '1', '2', '3' and '5'.
- Most of the beneficiaries (84.5%) belong to race '1', followed by race '2', '3' and '5'.
- Very few beneficiaries (2.1%) belong to race '5'.

2.3. EDA of Beneficiary Dataset — RenalDiseaseIndicator Feature:

Renal Disease Indicator is a flag to indicate whether the beneficiary has any problem related to kidney failure or not.

[Press enter or click to view image in full size](#)

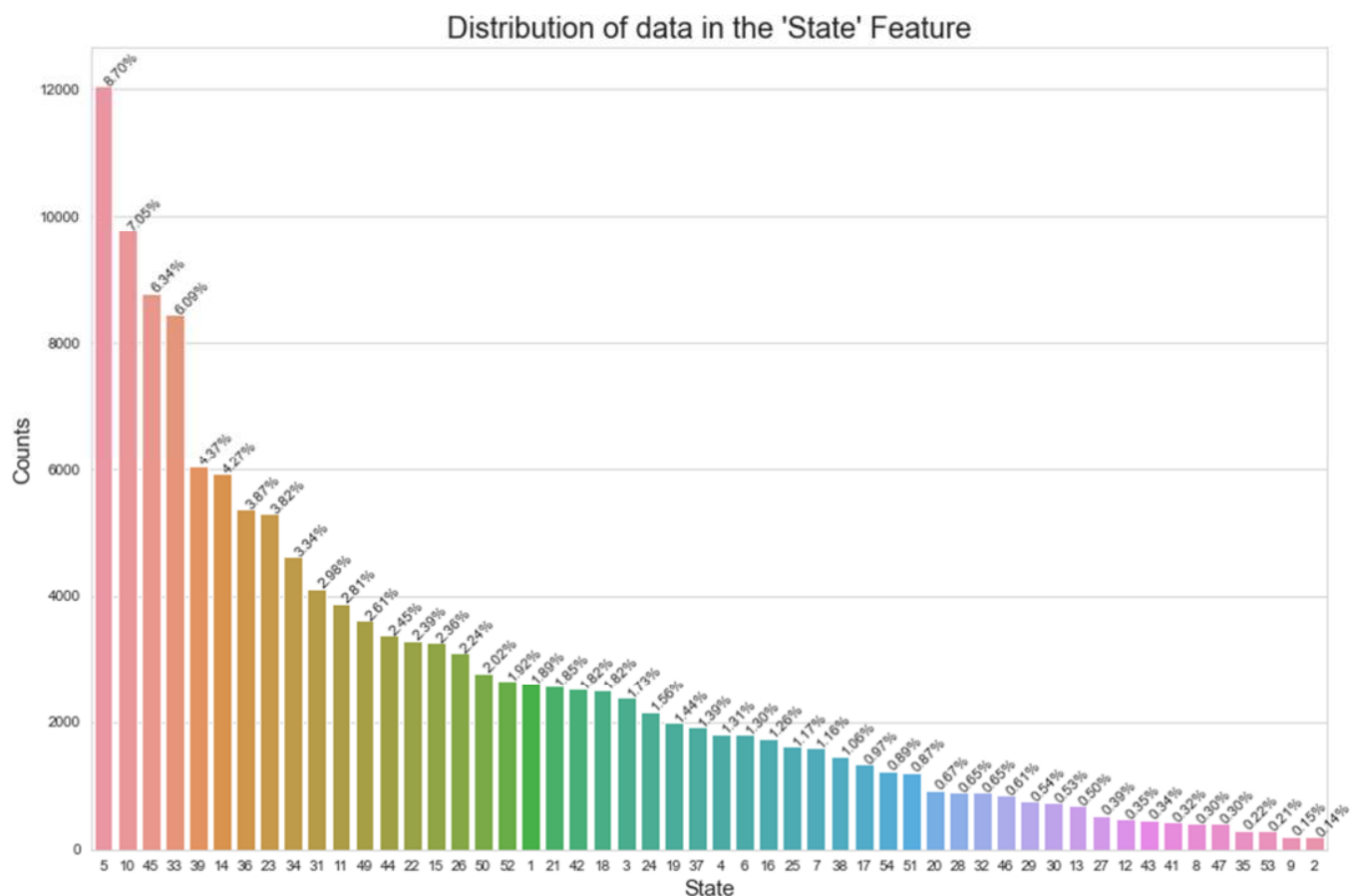


Observation:

- As this feature is a flag to indicate any kind of renal disease, there are two possible values: 0/No and 'Y'/Yes.
- Most of the beneficiaries (85.87%) do not have any renal disease.

2.4. EDA of Beneficiary Dataset — State Feature:

Press enter or click to view image in full size

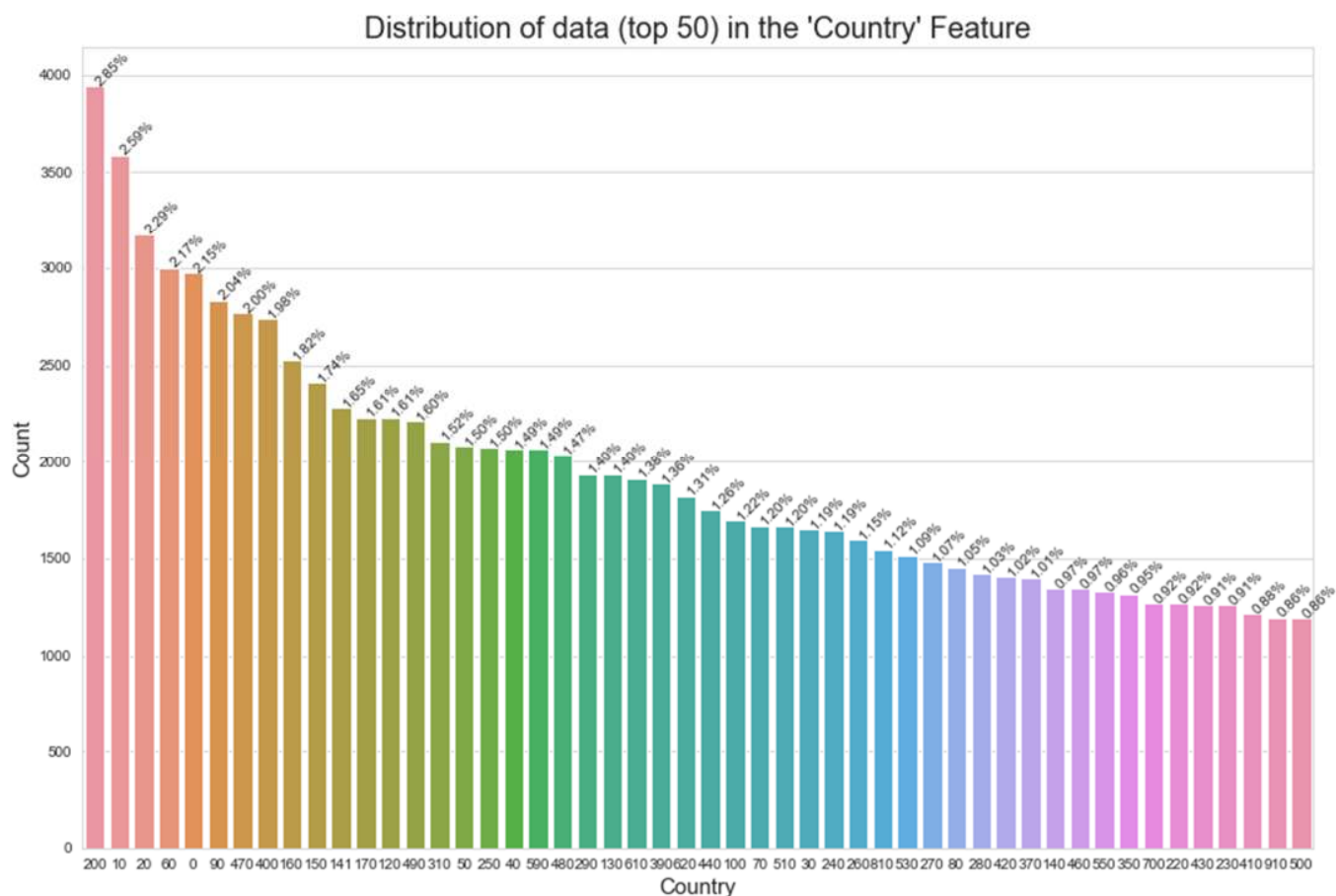


Observation:

- There are a total of 52 states.
- State with maximum beneficiaries (8.70%) is state encoded as '5'.
- State with minimum beneficiaries (0.14%) is state encoded as '2'.

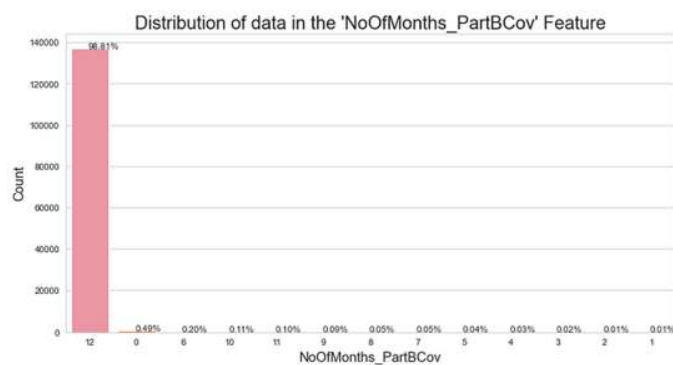
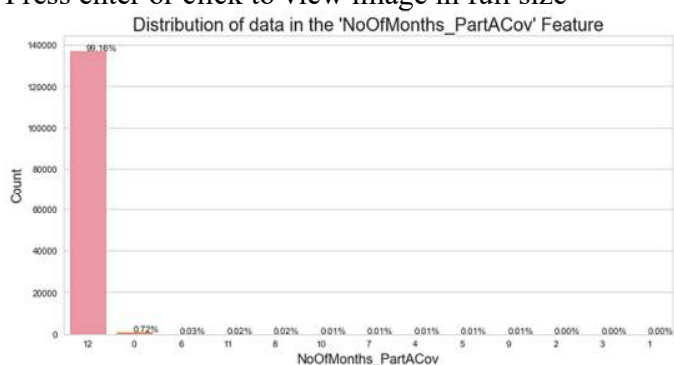
2.5. EDA of Beneficiary Dataset — Country Feature:

There are a total of 314 countries in the dataset. Plotting the distribution of data for the top 50 countries.
 Press enter or click to view image in full size



Observation:

- Countries having maximum beneficiaries are in the order: 200, 10, 20, and so on.
- Countries encoded as '200' have 2.85% of the beneficiaries.
- 2.6. EDA of Beneficiary Dataset — NoOfMonths_PartACov and NoOfMonths_PartBCov Features:**
- Press enter or click to view image in full size

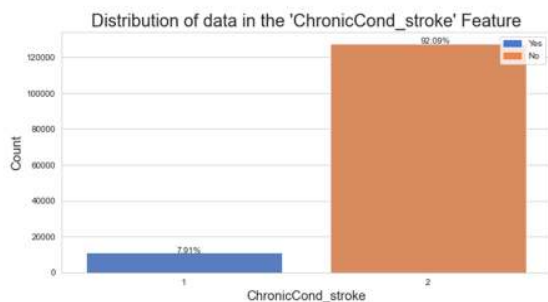
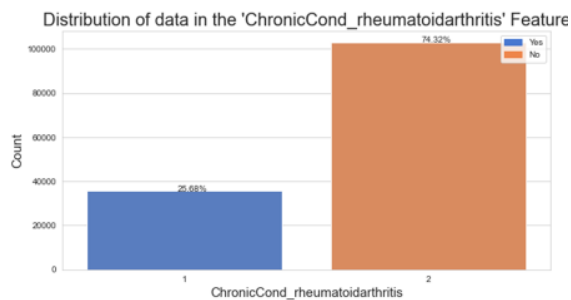
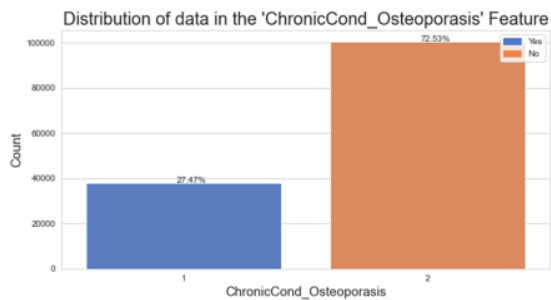
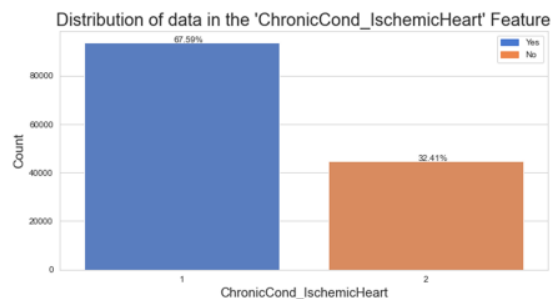
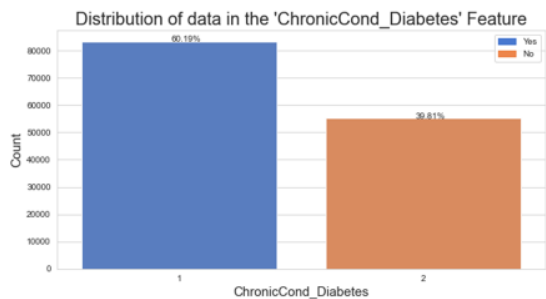
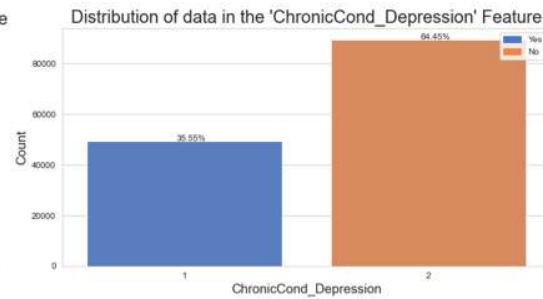
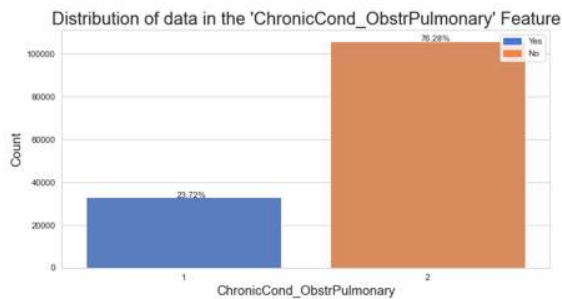
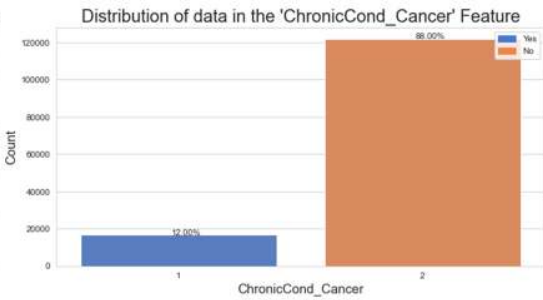
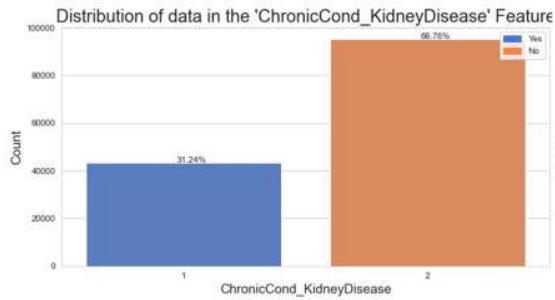
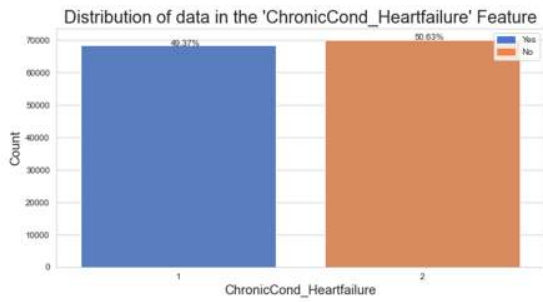
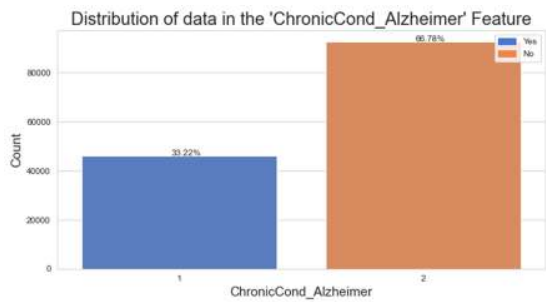


Observation:

- 99.16% of the data in the 'NoOfMonths_PartACov' feature has a value of '12' and 0.72% of the data has a value of '0'.
- 98.81% of the data in the 'NoOfMonths_PartBCov' feature has a value of '12'.

- All other values are negligible.
- As almost all values of this feature is having a value of 12, this feature won't make any impact in doing the classification. Hence, we can drop these features.

2.7. EDA of Beneficiary Dataset — Chronic Condition Features:

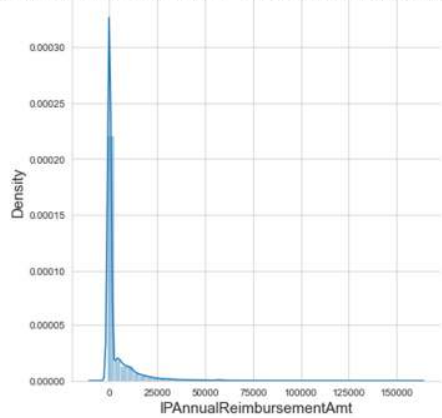


Observation:

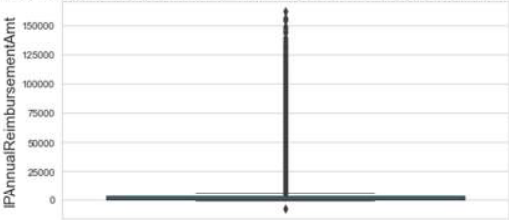
- There are two values in each of the Chronic Condition features: '1' and '2'.
- The value of '2' will be replaced with '0' to indicate '0' as 'No' and '1' as 'Yes' later during the featurization.
- The data is reasonably balanced for all features except for 'ChronicCond_Cancer' and 'ChronicCond_stroke' where the data is imbalanced.

2.8. EDA of Beneficiary Dataset — Annual Reimbursement and Deductible Amount Features for both Inpatient and Outpatient:

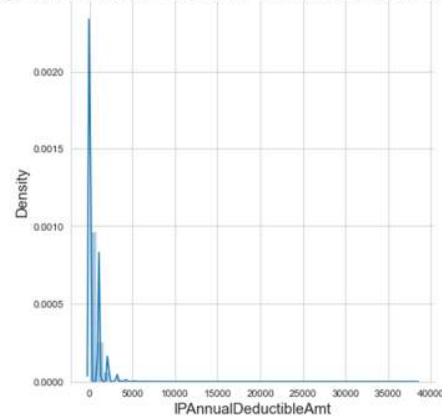
Histogram/PDF of the data in the 'IPAnnualReimbursementAmt' Feature



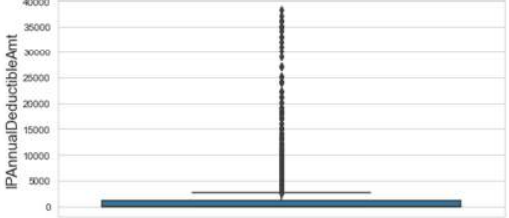
BoxPlot of the data in the 'IPAnnualReimbursementAmt' Feature



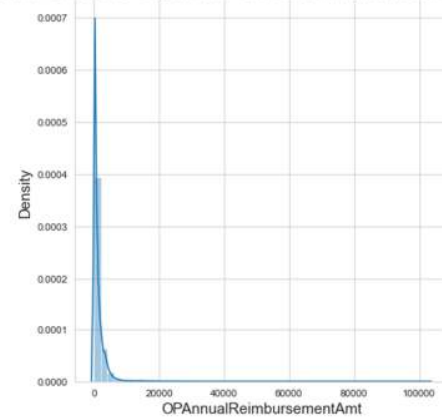
Histogram/PDF of the data in the 'IPAnnualDeductibleAmt' Feature



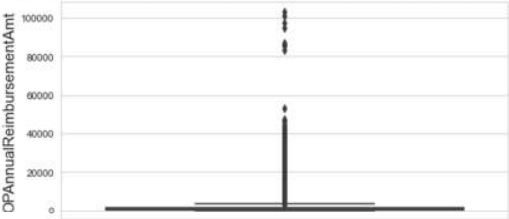
BoxPlot of the data in the 'IPAnnualDeductibleAmt' Feature



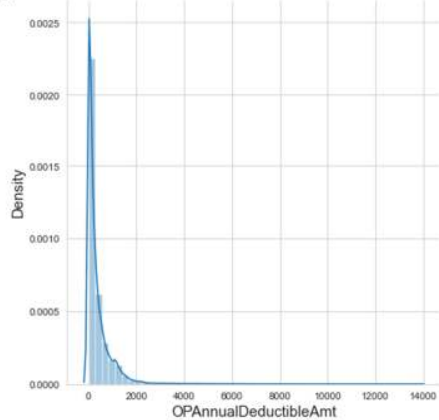
Histogram/PDF of the data in the 'OPAnnualReimbursementAmt' Feature



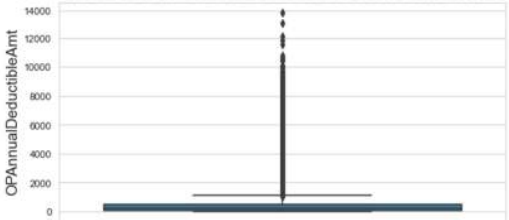
BoxPlot of the data in the 'OPAnnualReimbursementAmt' Feature



Histogram/PDF of the data in the 'OPAnnualDeductibleAmt' Feature



BoxPlot of the data in the 'OPAnnualDeductibleAmt' Feature

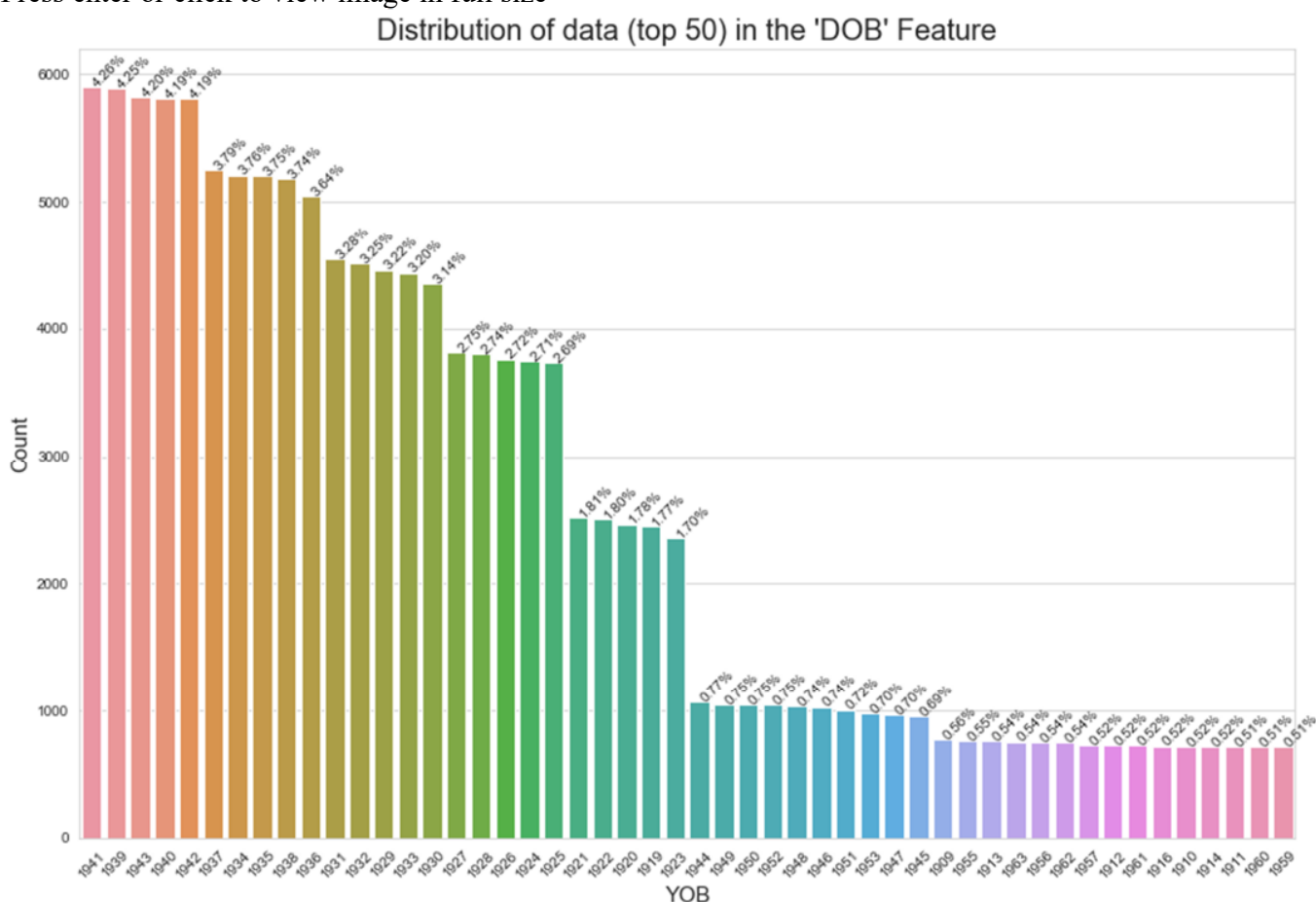


Observation:

- For IPAnnualReimbursementAmt Feature, most of the amount lies between 0 and 5000.
- For IPAnnualDeductibleAmt Feature, most of the amount lies between 0 and 2000.
- For OPAnnualReimbursementAmt Feature, most of the amount lies between 0 and 5000.
- For OPAnnualDeductibleAmt Feature, most of the amount lies between 0 and 2000.
- Rest all the amounts greater than the above ranges are outliers. In this case study, those high amount values can be the potential fraud cases. Hence, we won't remove those outliers.

2.9. EDA of Beneficiary Dataset — Date of Birth Feature:

Press enter or click to view image in full size

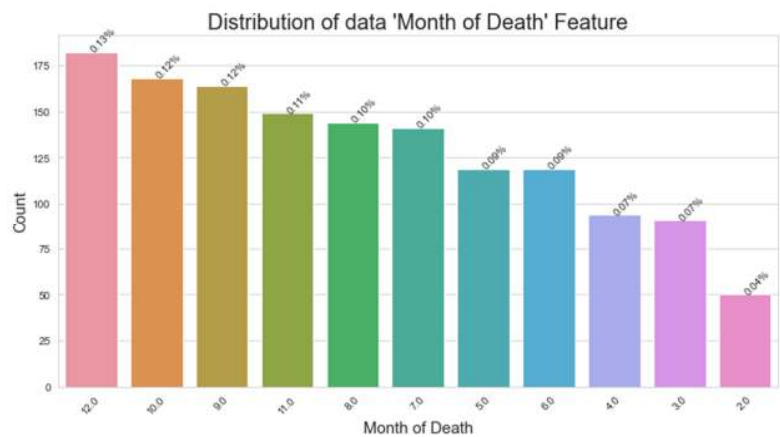
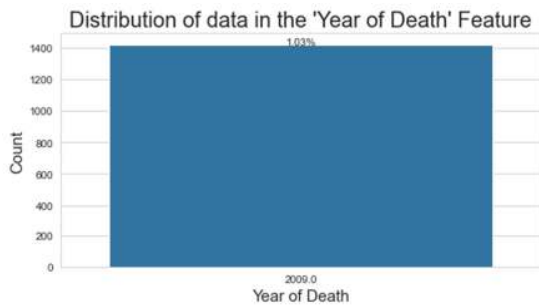


Observation:

- Most of the patients were born in the year 1941, followed by the years: 1939, 1943, 1940, and so on.

2.10. EDA of Beneficiary Dataset — Date of Death Feature:

Press enter or click to view image in full size



Observation:

- All the dead patients died in the year '2009'.
- Most of the dead patients died in December month.

3. EDA of Inpatient and Outpatient Dataset

Inpatient Dataset has a total of 30 features and Outpatient Dataset has a total of 27 features.

All the 27 features of Outpatient are also present in the Inpatient Dataset.

Inpatient Dataset has 3 extra features viz., 'AdmissionDt', 'DischargeDt' and 'DiagnosisGroupCode'.

Hence, we will merge the data from the Inpatient and Outpatient Dataset along with the common columns and do the EDA on the merged data.

3.1. EDA of Claim (Inpatient and Outpatient) Dataset — Date Features:

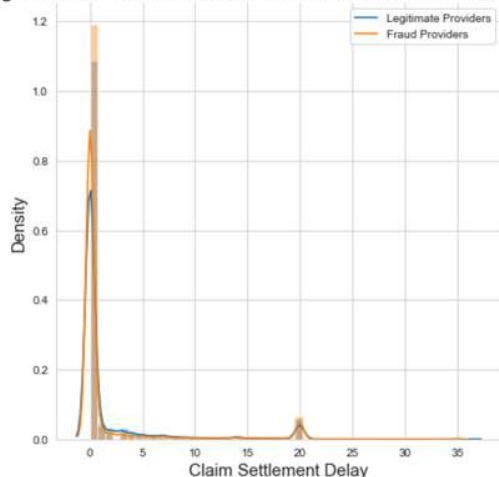
There are four date features in the Claim dataset viz., ClaimStartDt, ClaimEndDt, AdmissionDt and DischargeDt.

We will derive the below new features from these date features to do EDA:

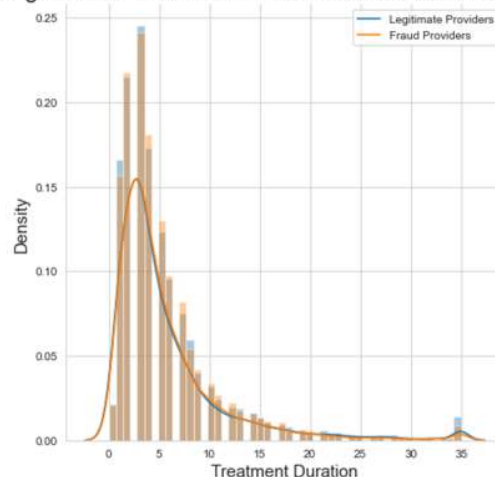
1. Claim Settlement Delay: Difference between the Claim End Date and Claim Start Date.
2. Treatment Duration: Difference between Discharge Date and Admission Date.

Press enter or click to view image in full size

Histogram/PDF of the data in the 'ClaimSettlementDelay' Feature



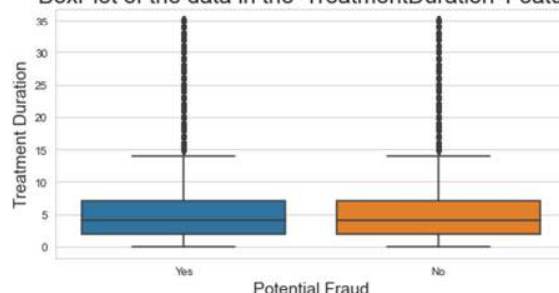
Histogram/PDF of the data in the 'TreatmentDuration' Feature



BoxPlot of the data in the 'ClaimSettlementDelay' Feature



BoxPlot of the data in the 'TreatmentDuration' Feature



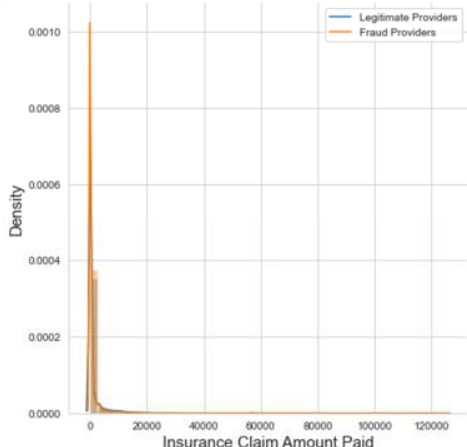
Observation:

- The distribution of data in both the 'ClaimSettlementDelay' and 'TreatmentDuration' features follow a similar pattern. The 'TreatmentDuration' is present only for Inpatient data and not for Outpatient data.
- Hence, we can keep 'ClaimSettlementDelay' and **discard the 'TreatmentDuration' feature**.
- Also, the data distribution for both classes (fraud and non-fraud) for each feature follows the same trend.
- There are outliers in the feature for both fraudulent and non-fraudulent cases, which should not be removed because those could be critical factors to identify a claim record as fraudulent or not.

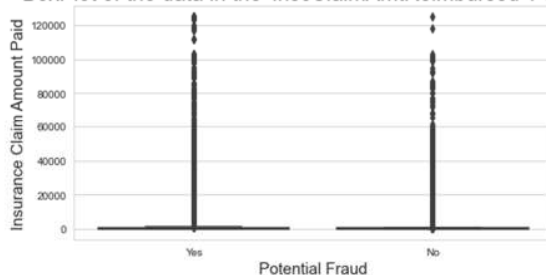
3.2. EDA of Claim (Inpatient and Outpatient) Dataset — Amount Features:

Press enter or click to view image in full size

Histogram/PDF of the data in the 'InscClaimAmtReimbursed' Feature



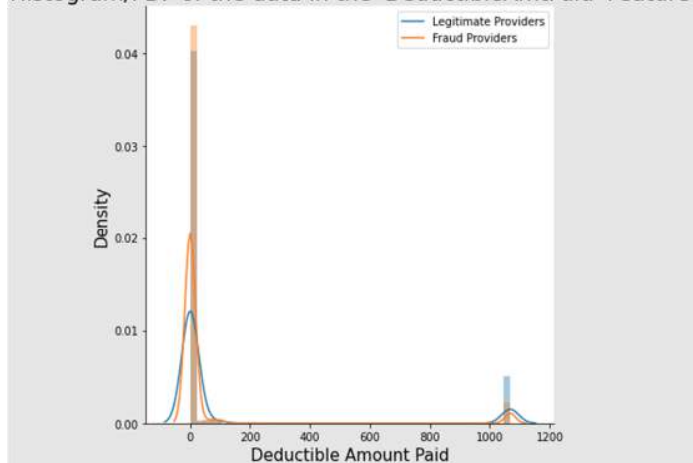
BoxPlot of the data in the 'InscClaimAmtReimbursed' Feature



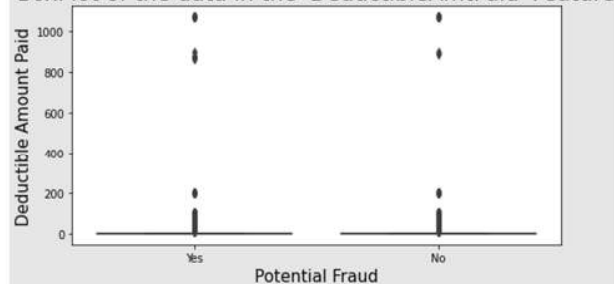
Observation:

- From the PDF, we can see that the data distribution of positive class and negative class follow an almost similar distribution.
- From the Box Plot, we can see that there are many outliers as well for both fraudulent and non-fraudulent classes.
- However, we will not remove the outliers because this feature is an important feature for a claim, and the records corresponding to the outliers can be classified as a potential fraud case when this feature is clubbed with other features to do the classification.

Histogram/PDF of the data in the 'DeductibleAmtPaid' Feature



BoxPlot of the data in the 'DeductibleAmtPaid' Feature



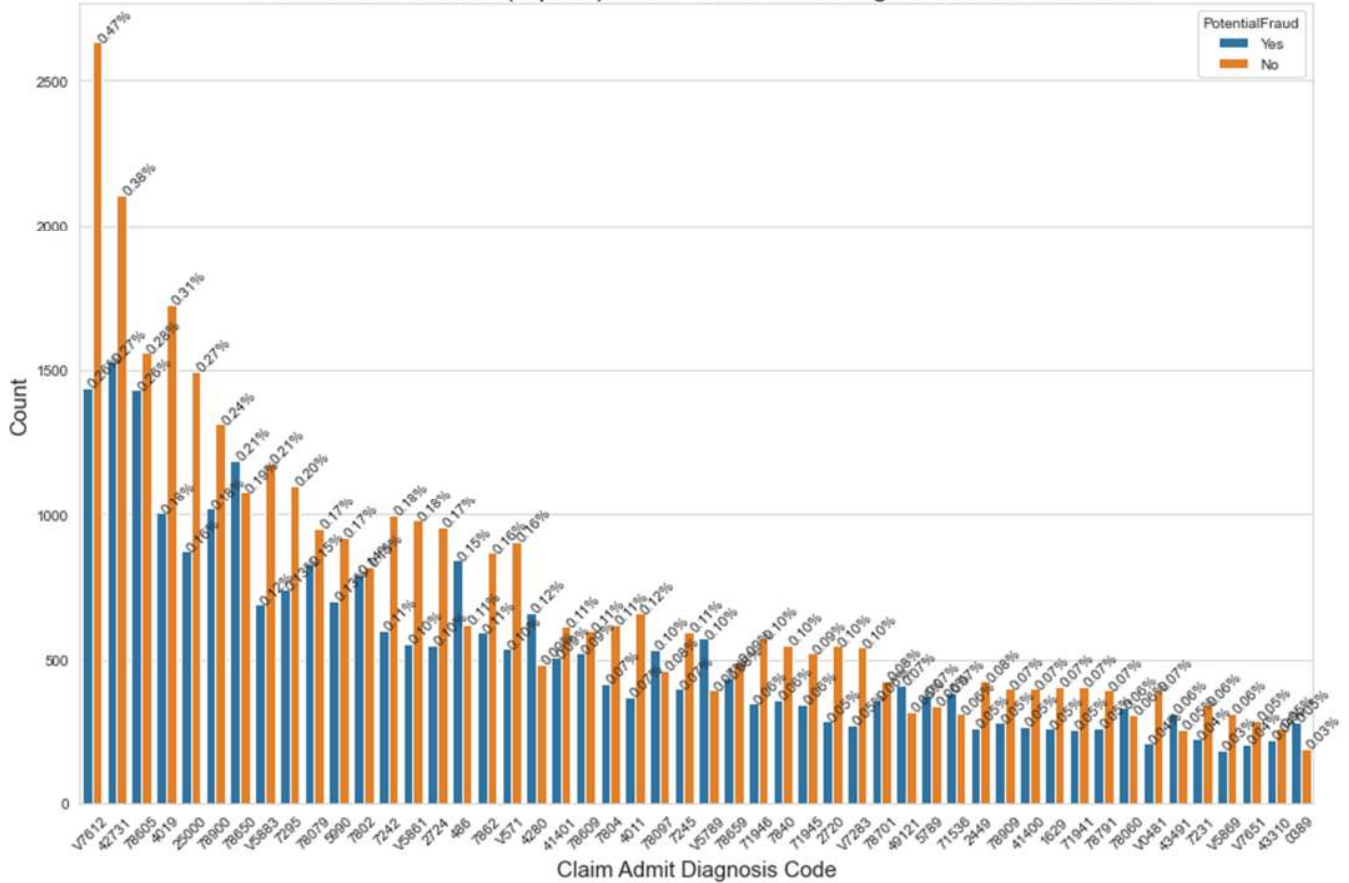
Observation:

- For the 'DeductibleAmtPaid' feature, most of the amount values are zero.
- Because of the too many zero values, we see the non-zero values as outliers in the Box Plot.

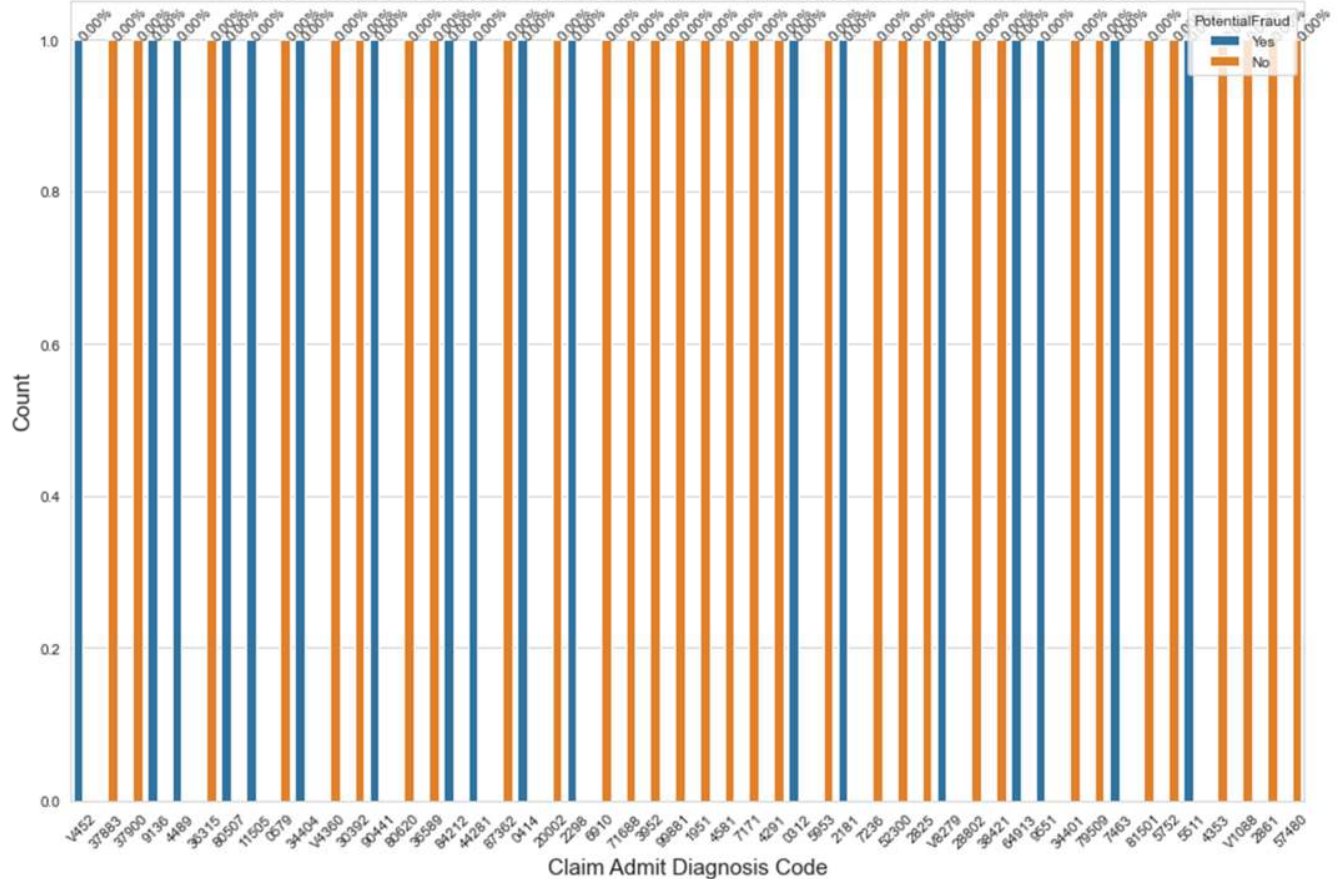
3.3. EDA of Claim (Inpatient and Outpatient) Dataset — ClmAdmitDiagnosisCode Feature:

Press enter or click to view image in full size

Distribution of data (top 50) in the 'ClnAdmitDiagnosisCode' Feature



Distribution of data (bottom 50) in the 'ClnAdmitDiagnosisCode' Feature



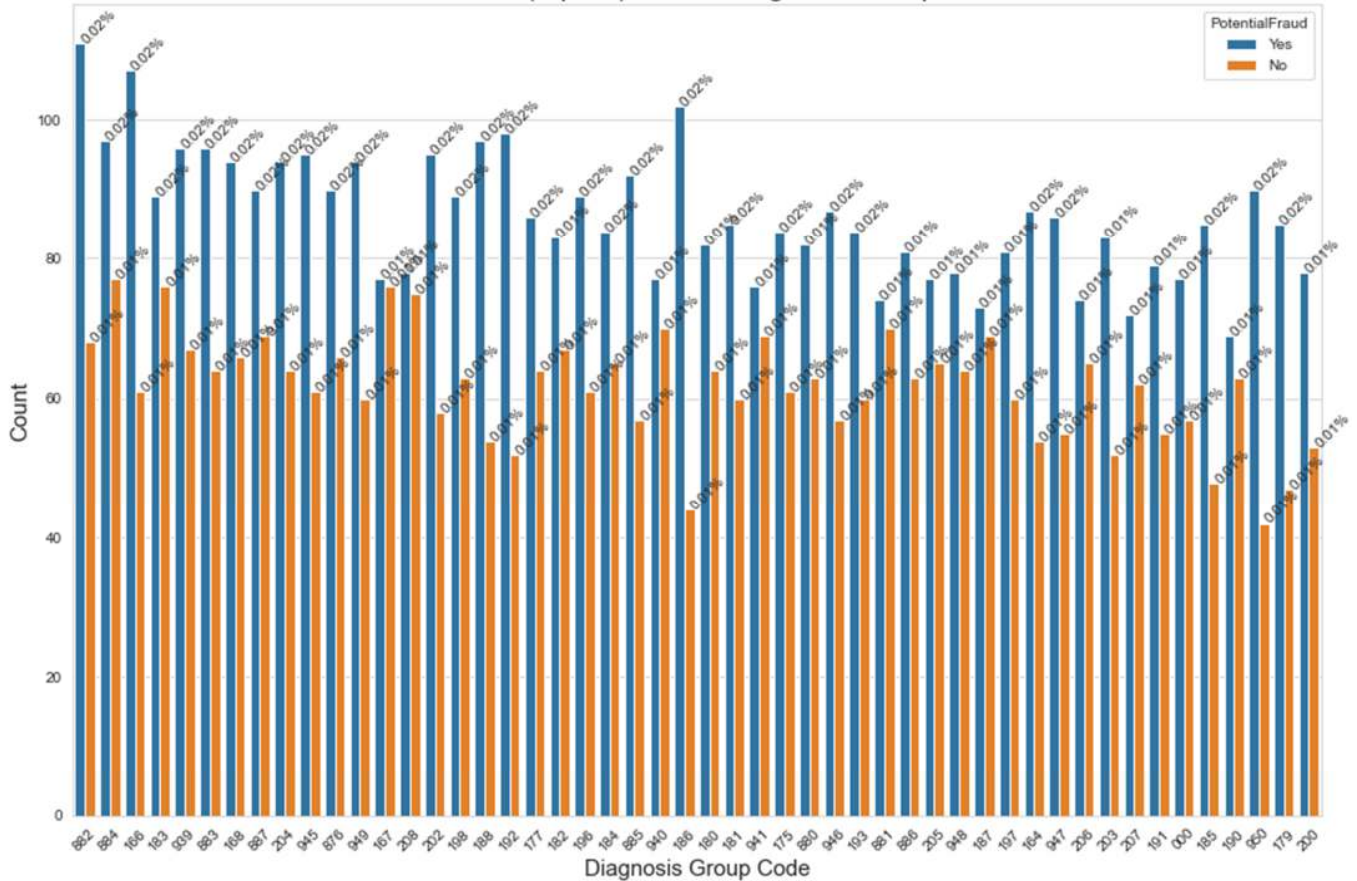
Observation:

- From the top 50 categories of the 'ClmAdmitDiagnosisCode' feature, we can see that all the codes (categories) are present for both positive (fraud) and negative (non-fraud) cases.
- From the bottom 50 categories of the 'ClmAdmitDiagnosisCode' feature, we can see that some codes (categories) are not present for both the classes but are present for either of them. For example, category 'V452' is present only once for fraud cases and category '37883' is present only once for the non-fraud case.
- The top 5 Claim Admit Diagnosis Codes are 'V7612', '42731', '78605', '4019' and '25000'.

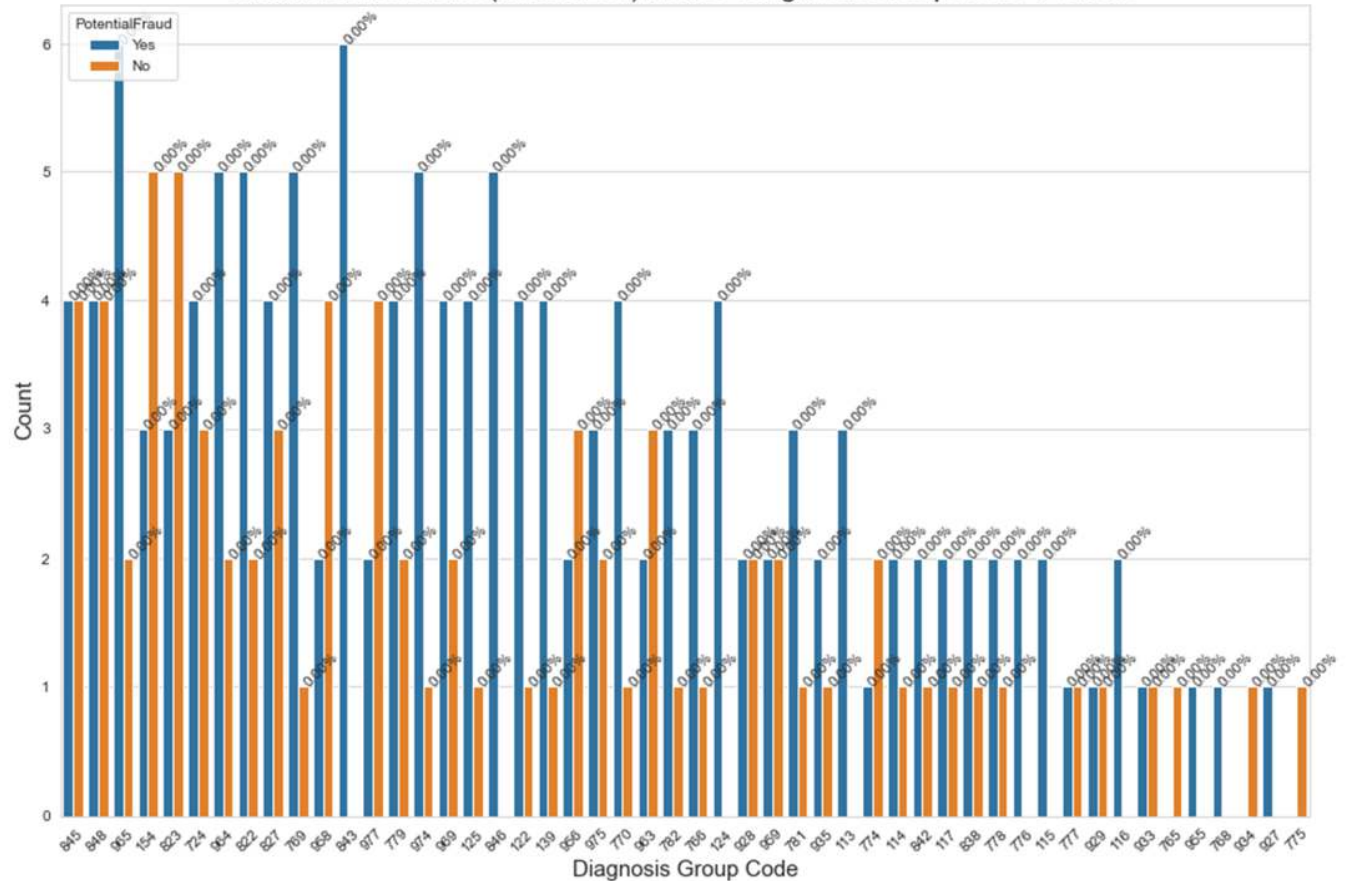
3.4. EDA of Claim (Inpatient) Dataset — DiagnosisGroupCode Feature:

Press enter or click to view image in full size

Distribution of data (top 50) in the 'DiagnosisGroupCode' Feature



Distribution of data (bottom 50) in the 'DiagnosisGroupCode' Feature



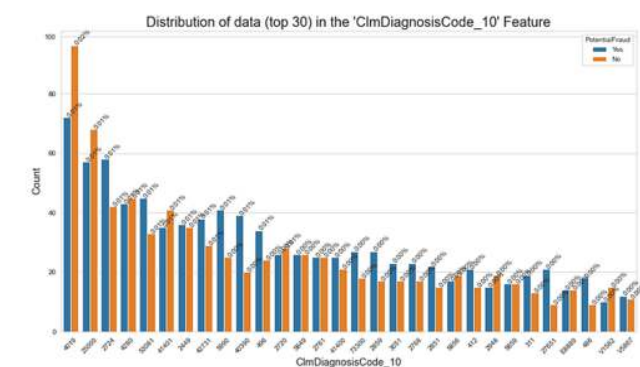
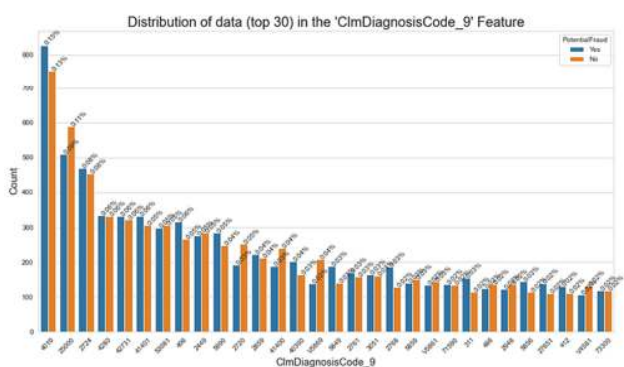
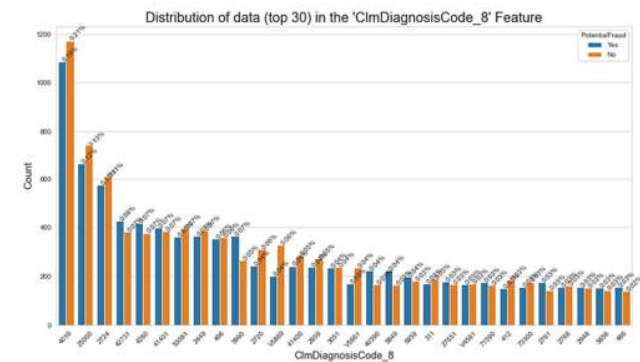
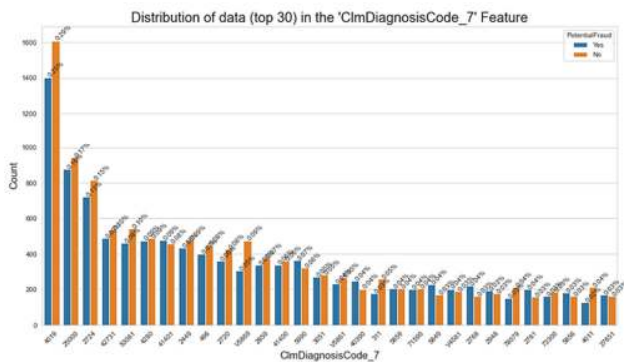
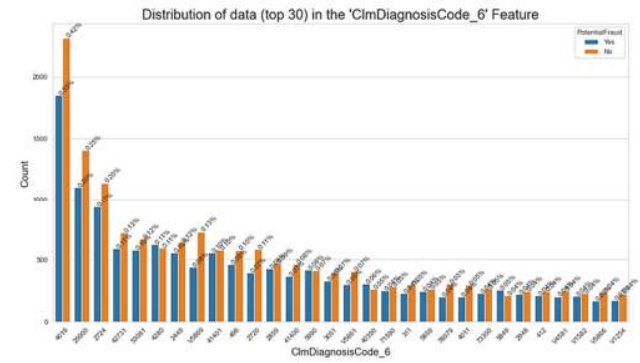
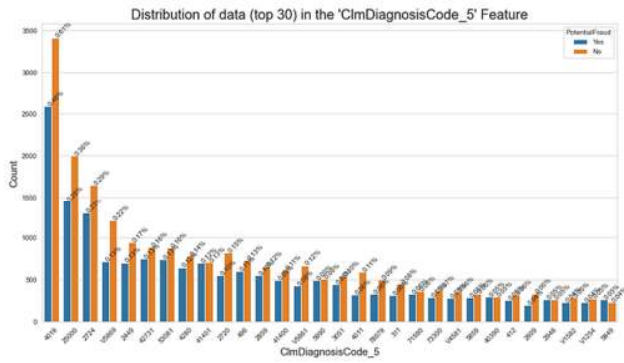
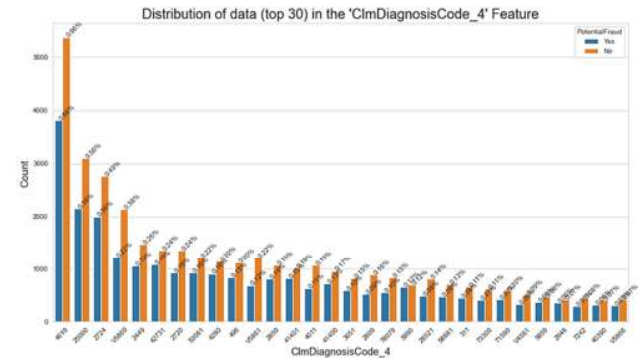
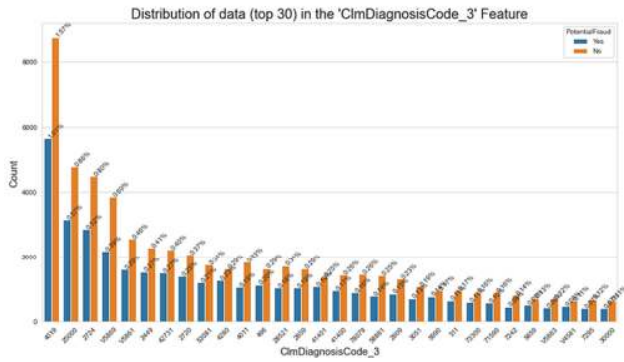
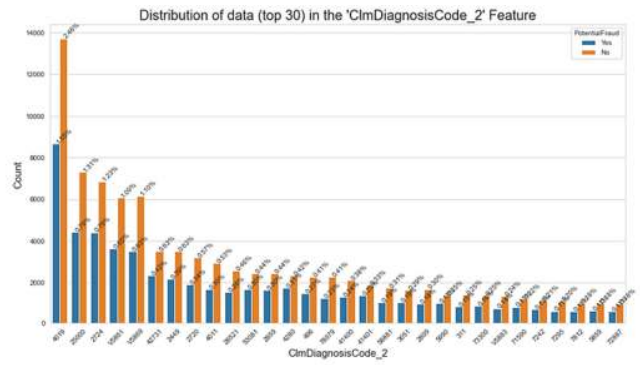
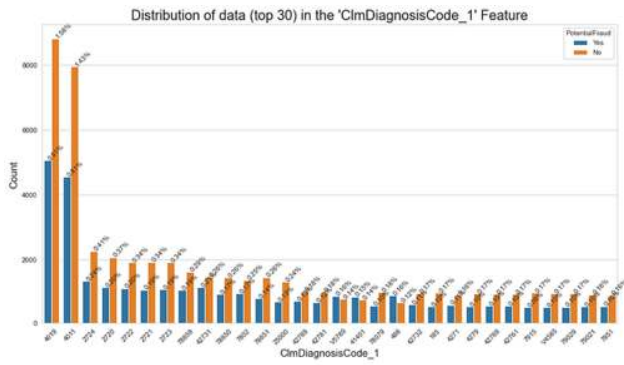
Observation:

- From the top 50 categories of the 'DiagnosisGroupCode' feature, we cannot deduce much information.
- From the bottom 50 categories of the 'DiagnosisGroupCode' feature, we can see that there are few codes that occur only once (Count = 1). In the 'ClmAdmitDiagnosisCode' feature, there are comparatively lot of codes that had only one occurrence.
- We cannot deduce much meaningful insight from this feature.

3.5. EDA of Claim (Inpatient and Outpatient) Dataset — Claim Diagnosis Codes Features:

Here, we will do the EDA for all the ten Claim Diagnosis Code Features.

- i. ClmDiagnosisCode_1: Diagnosis code identifying the beneficiary's principal diagnosis.
 - ii. ClmDiagnosisCode_2–10: Diagnosis code in the 2nd, 3rd, and so on, till the 10th position identifying the condition(s) for which the beneficiary is receiving care.
- Press enter or click to view image in full size

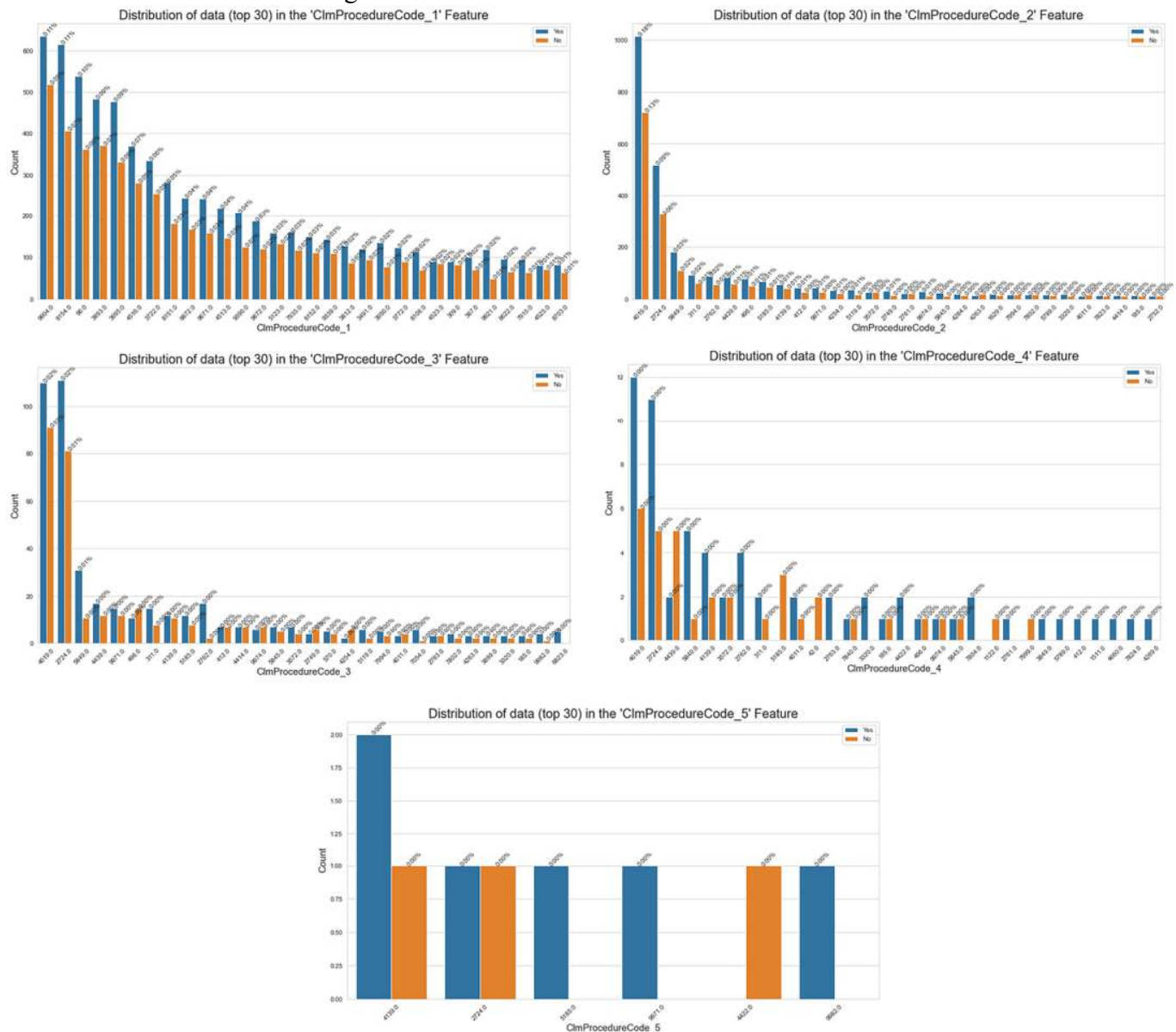


Observation:

- In all the Diagnosis Codes, code '4019' has the highest percentage of claims.
- In 'ClmDiagnosisCode_1', the 2nd and 3rd most occurrent codes are '4011' and '2724' respectively.
- In all features except 'ClmDiagnosisCode_1', the 2nd and 3rd most occurrent codes are '25000' and '2724' respectively.
- The codes '4019', '2724' and '42731' occur in all features.

3.6. EDA of Inpatient Dataset — Claim Procedure Codes Features:

Press enter or click to view image in full size

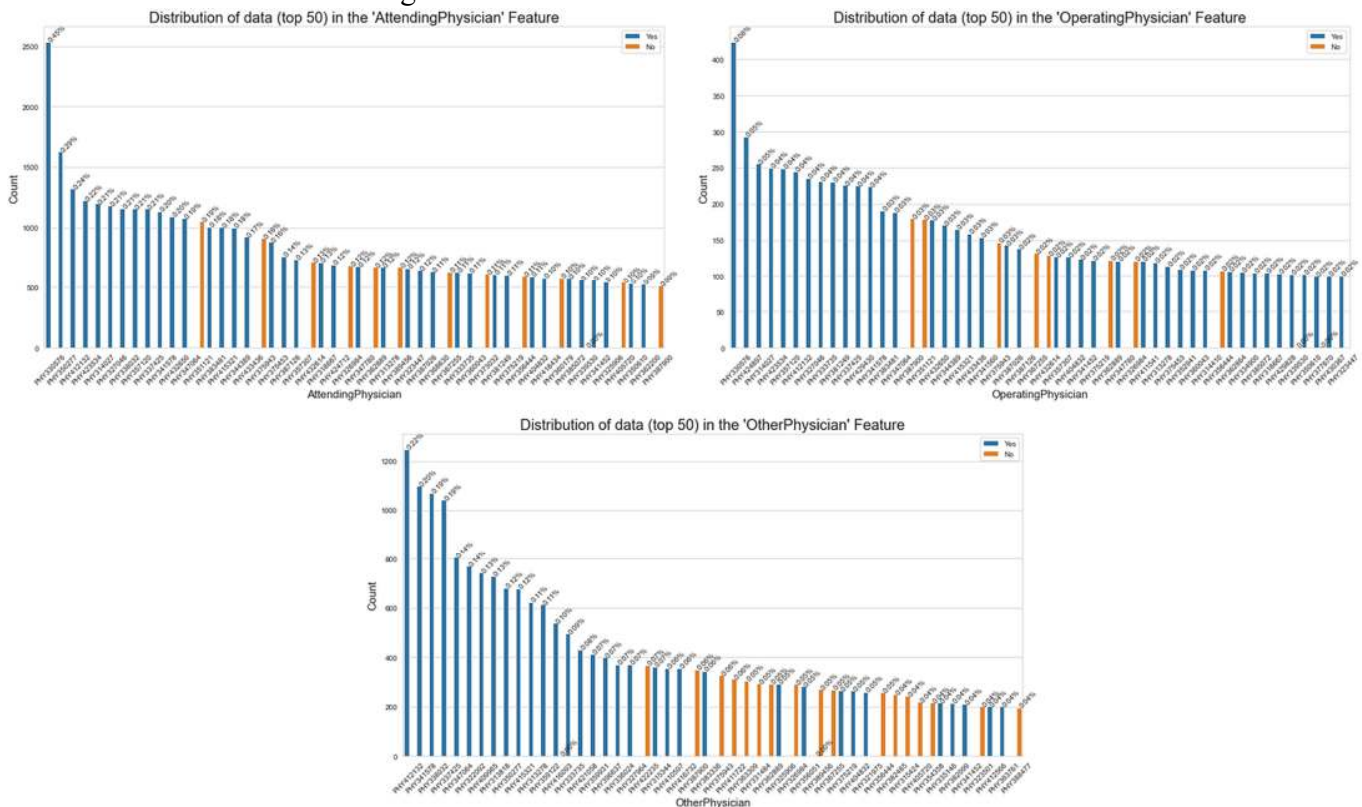


Observation:

- For 'ClmProcedureCode_1', code '9904.0' has the most occurrence, followed by '8154.0', '66.0', '3893.0', '3995', and so on. None of these 5 codes occur in any of the top 10 codes of the other 4 features.
- Code '4019.0' is most occurring code in the 3 features 'ClmProcedureCode_2', 'ClmProcedureCode_3' and 'ClmProcedureCode_4'.
- The most common codes in all the 5 features are '2724.0', '5185.0' and '4139.0' with four occurrences.
- Features 'ClmProcedureCode_4' and 'ClmProcedureCode_5' have very negligible (~0%) codes present and can be dropped.

3.7. EDA of Inpatient Dataset — Physician Features:

Press enter or click to view image in full size



Observation:

- In each of the plots of 'AttendingPhysician', 'OperatingPhysician' and 'OtherPhysician', it can be seen that the top Physicians (12+ in number) are involved in fraud cases only.
- Physician 'PHY330576' attended as well as operated on most of the patients.
- Around 2500+ patients were attended by the physician 'PHY330576'.
- Around 425 operations were performed by the physician 'PHY330576'.

- Physician 'PHY412132' was another physician for most of the patients.

Data Pre-processing

I. Merge the Datasets

Merge the given dataset as:

- Inpatient and Outpatient contain almost all similar columns and hence, can be concatenated. This would give a single dataset containing the details of both inpatient and outpatient details related to the claims.
- The resultant dataset can be combined with the Beneficiary dataset based on the Beneficiary Id to add the beneficiary-related information to the dataset.
- Finally, the dataset can be merged with the Provider dataset to map the class label to decide whether the Provider is fraudulent or not.

II. Data Preparation

Here, we will do clean-up, pre-processing, and feature engineering of the data.

1. Data Clean-up:

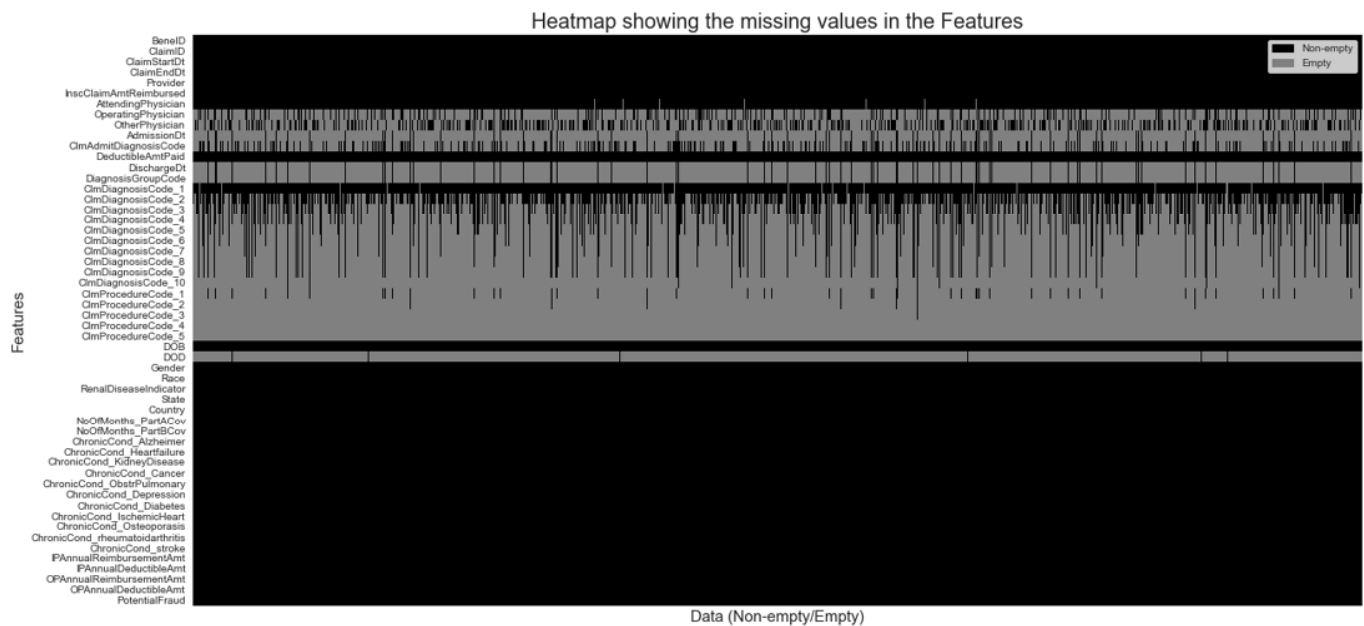
- Convert the features having binary values indicating flags like 'Y'/'N', 'Yes'/'No' and 1/2 to 1/0.
- Drop the columns having all null values.

2. Percentage of Null or Empty values:

- Check the percentage of null values for each column in the dataset.
- Also, plot a heatmap to see the percentage of missing values in the dataset.

Press enter or click to view image in full size

Press enter or click to view image in full size



Observation:

- Few of the Diagnosis Code and Procedural Codes are having a high percentage of empty values in the dataset.
- The variables having more than 99.5% empty cells are:
 1. ClmProcedureCode_3
 2. ClmProcedureCode_4
 3. ClmProcedureCode_5
- These columns can be removed as they have almost all empty cells and won't contribute much to the classification.

3. Feature Engineering:

3.1. New Features from Date Features:

- Here, we will generate new features from the date features:
 1. Claim Settlement Delay: Difference between the Claim End Date and Claim Start Date.
 2. Treatment Duration: Difference between Discharge Date and Admission Date.
 3. Age: Based on the DOB.
 4. IsDead: Based on the DOD.

3.2. New Features from Reimbursement Amount and Deductible Amount:

- Here, we will generate new features from the features:
InscClaimAmtReimbursed, DeductibleAmtPaid, IPAnnualReimbursementAmt, IPAnnualDeductibleAmt, OPAnnualReimbursementAmt and OPAnnualDeductibleAmt.
- $\text{TotalClaimAmount} = \text{InscClaimAmtReimbursed} + \text{DeductibleAmtPaid}$.
- $\text{IPTotalAmount} = \text{IPAnnualReimbursementAmt} + \text{IPAnnualDeductibleAmt}$.
- $\text{OPTotalAmount} = \text{OPAnnualReimbursementAmt} + \text{OPAnnualDeductibleAmt}$.

3.3. New Features to indicate Inpatient and Outpatient:

- Here, we will generate a new feature called 'IsInpatient' having a value of 1 if the claim record is for Inpatient data and 0 if the claim record is for Outpatient data.
- As the feature 'DiagnosisGroupCode' exists only for the Inpatient dataset, we will create the values for 'IsInpatient' based on whether is a value in the 'DiagnosisGroupCode' feature.

3.4. New Features from the Physician related features:

- We will derive new features from the features 'AttendingPhysician', 'OperatingPhysician' and 'OtherPhysician' as:
 1. UniquePhysCount: Number of different physicians who attended/operated the patient and/or performed any other work in a claim record.
 2. PhysRoleCount: Total number of attending, operating and other physicians who worked for a patient in a claim record.
 3. IsSamePhysMultiRole1: Flag to indicate if a single physician is involved with multiple roles for a patient in a claim record.
 4. IsSamePhysMultiRole2: Flag to indicate if there are only two physicians involved in a claim record and one of them had multiple roles.
- Consider the below table data to understand it better:

Press enter or click to view image in full size

AttendingPhysician	OperatingPhysician	OtherPhysician	UniquePhysCount	PhysRoleCount	IsSamePhysMultiRole1	IsSamePhysMultiRole2
PHY01	PHY02	PHY03	3	3	0	0
PHY01	PHY02	-	2	2	0	0
PHY01	-	-	1	1	0	0
PHY01	PHY01	-	1	2	1	0
PHY01	PHY01	PHY01	1	3	1	0
PHY01	PHY01	PHY02	2	3	0	1

- Will create 3 new features for the top 3 physicians and count the number of times they appeared as attending, operating or other physicians in a claim record.
- If 'PHY412132', 'PHY337425' and 'PHY330576' are the top 3 most occurring physicians, consider the below table data to understand the new features:

Press enter or click to view image in full size

AttendingPhysician	OperatingPhysician	OtherPhysician	PHY412132	PHY337425	PHY330576
PHY412132	PHY337425	PHY337425	1	2	0
PHY330576	PHY412132	-	1	0	1
PHY330576	-	-	0	0	1
PHY337425	PHY337425	-	0	2	0
PHY412132	PHY412132	PHY412132	3	0	0
PHY330576	PHY337425	PHY412132	1	1	1

3.5. New Features from the Claim Diagnosis Codes and Claim Procedure Codes:

- For all the features related to the Claim Diagnosis Codes and Claim Procedure Codes, we would generate new features for the top 'k' codes similar to what we did for the top 3 physicians above.

3.6. Update the ClmAdmitDiagnosisCode and DiagnosisGroupCode Feature:

- Replace the value of the features with a flag to indicate whether a value exists or not.

3.7. New Features for State and Country:

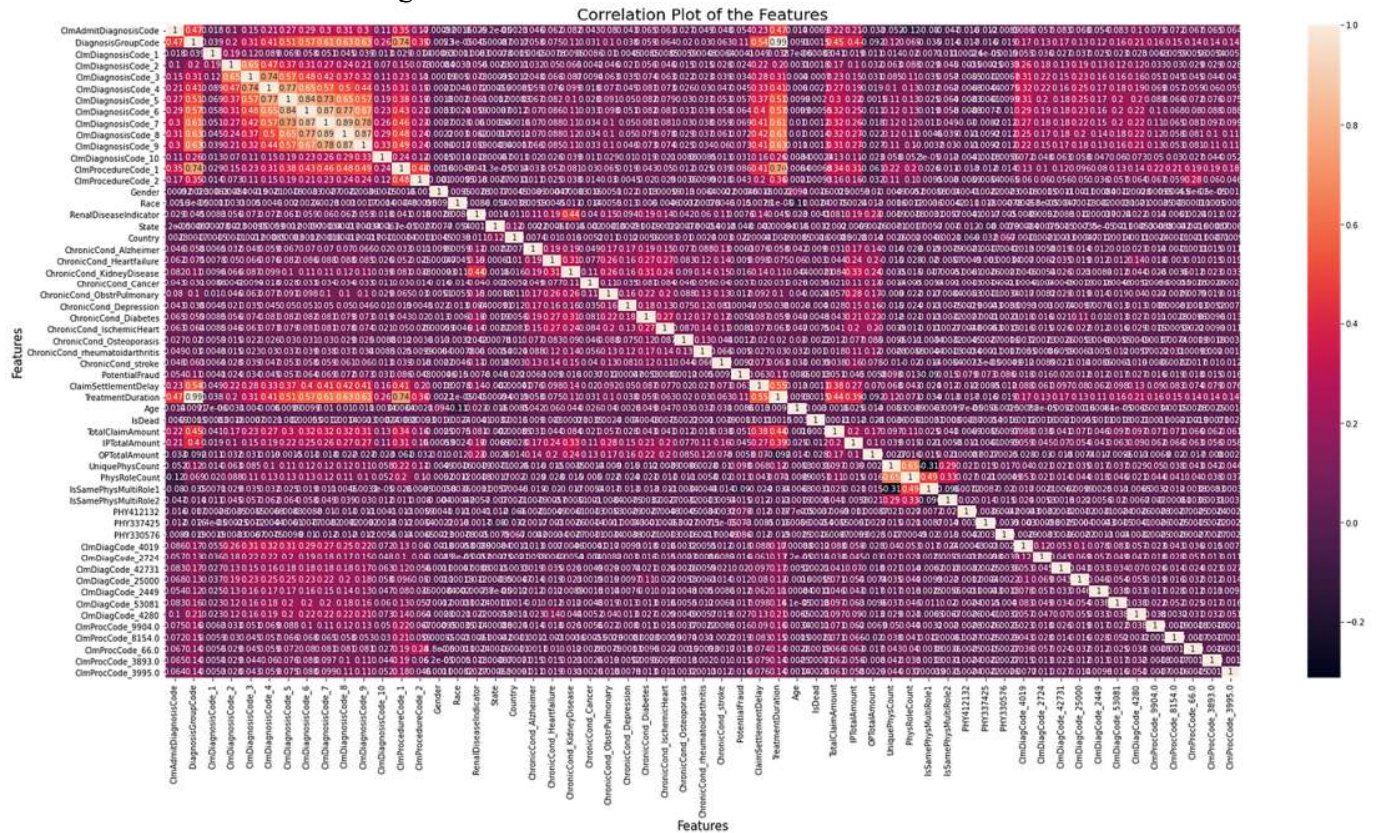
- We will do the below two kinds of encoding for the 'State' and 'Country' features, after splitting the data into train, test, and CV:
 1. **Response Encoding:** will be used in the models suitable for lower dimensions like Decision Tree, Random Forest, GBDT, etc.
 2. **One-hot Encoding:** for a model suitable for handling large dimensions as well. Eg., Naive Bayes, Logistic Regression, etc.

4. Correlation Plot:

- We will plot the **Spearman Rank Correlation Coefficient** Graph for the final dataset prepared and see if any features can be removed based on the Correlation Coefficient.
- Will consider the correlation coefficient of threshold 0.70.
- Will check for the pair of features having a correlation coefficient greater than 0.70.

- Out of the two features in a pair, we will remove the feature having a lesser correlation coefficient with the Class Label 'PotentialFraud'.

Press enter or click to view image in full size



Observation from the Correlation Plot:

- Here, we see few features are having high Spearman Correlation Coefficient.
- Spearman Correlation Coefficient among some of the features having coefficient greater than 0.70 are:

- TreatmentDuration and DiagnosisGroupCode: 0.99
- TreatmentDuration and CmlProcedureCode_1: 0.74
- DiagnosisGroupCode and CmlProcedureCode_1: 0.74
- CmlDiagnosisCode_3 and CmlDiagnosisCode_4: 0.74
- CmlDiagnosisCode_4 and CmlDiagnosisCode_5: 0.77
- CmlDiagnosisCode_5 and CmlDiagnosisCode_6: 0.84
- CmlDiagnosisCode_5 and CmlDiagnosisCode_7: 0.73
- CmlDiagnosisCode_6 and CmlDiagnosisCode_7: 0.87
- CmlDiagnosisCode_6 and CmlDiagnosisCode_8: 0.77

10. ClmDiagnosisCode_7 and ClmDiagnosisCode_8: 0.89

11. ClmDiagnosisCode_7 and ClmDiagnosisCode_9: 0.78

12. ClmDiagnosisCode_8 and ClmDiagnosisCode_9: 0.87

Removal of Features:

- Will remove one of the features among these pairs that has a lesser correlation coefficient with the Class Label 'PotentialFraud'.
- The Correlation Coefficient of the Feature with the Class Label is shown in parenthesis.
- Removing the below feature among the pair of features based on the correlation coefficient with the Class Label:
 1. TreatmentDuration (0.11) and DiagnosisGroupCode (0.11): Any feature can be removed as both have a correlation coefficient of 0.11 with the Class Label. Remove the feature 'DiagnosisGroupCode' here.
 2. TreatmentDuration (0.11) and ClmProcedureCode_1 (0.086): Remove the feature 'ClmProcedureCode_1'.
 3. DiagnosisGroupCode (0.11) and ClmProcedureCode_1 (0.086): Both these features are already accounted for removal.
 4. ClmDiagnosisCode_3 (0.034) and ClmDiagnosisCode_4 (0.045): Remove the feature 'ClmDiagnosisCode_3'.
 5. ClmDiagnosisCode_4 (0.045) and ClmDiagnosisCode_5 (0.057): Remove the feature 'ClmDiagnosisCode_4'.
 6. ClmDiagnosisCode_5 (0.057) and ClmDiagnosisCode_6 (0.064): Remove the feature 'ClmDiagnosisCode_5'.
 7. ClmDiagnosisCode_5 (0.057) and ClmDiagnosisCode_7 (0.069): 'ClmDiagnosisCode_5' has the lesser coefficient and is already accounted in the above point.
 8. ClmDiagnosisCode_6 (0.064) and ClmDiagnosisCode_7 (0.069): Remove the feature 'ClmDiagnosisCode_6'.
 9. ClmDiagnosisCode_6 (0.064) and ClmDiagnosisCode_8 (0.072): 'ClmDiagnosisCode_6' has the lesser coefficient and is already accounted in the above point.
 10. ClmDiagnosisCode_7 (0.069) and ClmDiagnosisCode_8 (0.072): Remove the feature 'ClmDiagnosisCode_7'.

11. ClmDiagnosisCode_7 (0.069) and ClmDiagnosisCode_9 (0.073): 'ClmDiagnosisCode_7' has the lesser coefficient and is already accounted in the above point.
12. ClmDiagnosisCode_8 (0.072) and ClmDiagnosisCode_9 (0.073): Remove the feature 'ClmDiagnosisCode_8'.

Code for all the Data Pre-processing:

```
import numpy as np
import pandas as pd
```

```
def PreprocessData(xData):
```

```
    """
    Function to implement the data pipeline for transforming the dataset into the required format as required by
    the
    Model.
```

```
    Parameters:
```

```
    -----
```

```
    xData: DataFrame
```

```
        Dataset containing the features.
```

```
    """
```

```
    # Create a copy of the dataset
```

```
    data = xData.copy()
```

```
    #region - Data Cleanup-----
```

```
    #=====
```

```
def dataCleanup(data):
```

```
    """
```

```
    Cleans up the data in the given dataset and returns the cleaned dataset.
```

```
    Parameter:
```

```
    -----
```

```
    data: pandas.core.frame.DataFrame
```

```
        Dataframe to be cleaned up
```

```
    """
```

```
    #region - Beneficiary Columns-----
```

```
    # 'Renal Disease Indicator' column.
```

```
    # RenalDiseaseIndicator column has two unique values: 0 and 'Y'. Replace the value of 'Y' with 1.
```

```
    data['RenalDiseaseIndicator'].replace(to_replace='Y', value=1, inplace=True)
```

```
    # Convert the datatype of the 'RenalDiseaseIndicator' feature to numeric.
```

```
    data['RenalDiseaseIndicator'] = data['RenalDiseaseIndicator'].apply(pd.to_numeric)
```

```

# 'Chronic Condition' columns
# 'ChronicCond_' columns contains two unique values: 1 and 2. Replace the value of 2 with 0 to indicate 1
as 'Yes'
# and 0 as 'No'
for chronicCol in [col for col in list(data.columns) if 'Chronic' in col]:
    data[chronicCol].replace(to_replace=2, value=0, inplace=True)

# Rename the column 'County' as 'Country'
data.rename(columns={'County': 'Country'}, inplace=True)

#endregion - Beneficiary Columns-----

#region - Inpatient and Outpatient Columns-----

# Fill empty values of the 'DeductibleAmtPaid' feature as it is a numeric feature and has some empty
values.
data['DeductibleAmtPaid'].fillna(value=0, inplace=True)

#region - Inpatient and Outpatient Columns-----

#region - Other columns -----

# Drop the columns having all null values
data.dropna(axis=1, how='all', inplace=True)

# Replace the class label 'PotentialFraud' values. Replace 'Yes' with 1 and 'No' with 0.
if ('PotentialFraud' in data.columns):

    data['PotentialFraud'].replace(to_replace=['Yes', 'No'], value=[1, 0], inplace=True)

#endregion - Other columns -----

# Call the function 'dataCleanup' to clean the data in the dataframe
dataCleanup(data)

# Set of columns to be removed
columnsToRemove = ['ClmProcedureCode_3', 'ClmProcedureCode_4', 'ClmProcedureCode_5']

# Remove the above set of columns from the dataframe.
data.drop(columns=columnsToRemove, inplace=True)

#endregion - Data Cleanup-----

#=====
=====

```

#region - Date Features-----

#=====

Date related columns

colDate = [col for col in data.columns if ('Dt' in col or 'DOB' in col or 'DOD' in col)]

Convert these columns to DateTime Format

data[colDate] = data[colDate].apply(pd.to_datetime)

Generate new Feature 'ClaimSettlementDelay' (ClaimEndDt - ClaimStartDt)

data['ClaimSettlementDelay'] = (data['ClaimEndDt'] - data['ClaimStartDt']).dt.days

Generate new Feature 'TreatmentDuration' (DischargeDt - AdmissionDt)

data['TreatmentDuration'] = (data['DischargeDt'] - data['AdmissionDt']).dt.days

data['TreatmentDuration'].fillna(0, inplace=True) # Filling empty values with 0 because the features
'DischargeDt' and 'AdmissionDt' exist only for Inpatient records.

data['TreatmentDuration'] = data['TreatmentDuration'].apply(int)

maxDate = max(data['ClaimEndDt'].max(), data['DischargeDt'].max())

Generate 'Age' feature from DOB based on the DOD or the maximum date.

data['Age'] = data.apply(lambda x: round(((x['DOD'] - x['DOB']).days)/365) if pd.notnull(x['DOD'])
else round(((maxDate - x['DOB']).days)/365), axis=1)

Generate new Feature 'IsDead' based on whether there is a value in the DOD column or not

data['IsDead'] = data['DOD'].apply(lambda x: 1 if pd.notnull(x) else 0)

Remove the set of date columns from the dataframe

data.drop(columns=colDate, inplace=True)

#endregion - Date Features-----

#=====

#region - Amount Features-----

#=====

```

# Fetch the features related to amount.
colAmt = [col for col in data.columns if 'Amt' in col]

# Total Claim Amount = Insurance Claim Amount reimbursed + Deductible Amount paid by the Subscriber
data['TotalClaimAmount'] = data['InscClaimAmtReimbursed'] + data['DeductibleAmtPaid']

# Total Inpatient Amount = Inpatient Annual Amount reimbursed + Inpatient Annual Deductible Amount
data['IPTotalAmount'] = data['IPAnnualReimbursementAmt'] + data['IPAnnualDeductibleAmt']

# Total Outpatient Amount = Outpatient Annual Amount reimbursed + Outpatient Annual Deductible
Amount
data['OPTotalAmount'] = data['OPAnnualReimbursementAmt'] + data['OPAnnualDeductibleAmt']

# Remove the set of old amount features from the dataframe
data.drop(columns=colAmt, inplace=True)

#endregion - Amount Features-----

#=====

# Fetch the Claim Id and Provider Id from the dataset
identifierData = data[['ClaimID', 'Provider']]

# Drop some of the unique identifiers from the dataset as they won't contribute anything to do the
classification.
data.drop(columns=['ClaimID', 'BeneID', 'Provider', 'NoOfMonths_PartACov', 'NoOfMonths_PartBCov'],
inplace=True)

#region - Physician Features-----

#=====

# Fetch the columns related to Physicians
colPhys = [col for col in data.columns if 'Physician' in col]

# Prepare the feature 'UniquePhysCount'
data['UniquePhysCount'] = data[colPhys].apply(lambda x: len(set([phys for phys in x if not
pd.isnull(phys)])), axis=1)

# Prepare the feature 'PhysRoleCount'
data['PhysRoleCount'] = data[colPhys].apply(lambda x: len([phys for phys in x if not pd.isnull(phys)]),
axis=1)

```



```

# Prepare the feature 'IsSamePhysMultiRole1'
data['IsSamePhysMultiRole1'] = data[['UniquePhysCount','PhysRoleCount']] \
    .apply(lambda x: 1 if x['UniquePhysCount'] == 1 and x['PhysRoleCount'] > 1 else 0,
axis=1)

```

```

# Prepare the feature 'IsSamePhysMultiRole2'
data['IsSamePhysMultiRole2'] = data[['UniquePhysCount','PhysRoleCount']] \
    .apply(lambda x: 1 if x['UniquePhysCount'] == 2 and x['PhysRoleCount'] > 2 else 0,
axis=1)

```

```

def encodeCatFeatures(dataset, existingFeatures, newFeatures, suffix=""):
    """
    Function to create new encoded features for Categorical Features, based on their count of values
    in the existing set of features.
    Parameters:
    -----
    dataset: pandas.core.frame.DataFrame
        DataFrame containing the data for which the new set of encoded features has to be created.
    existingFeatures: list
        List of existing features to considered for counting.
    newFeatures: list
        List of new features to encoded and created
    suffix: str
        Suffix to add before the new feature names.
    """

    # Fetch the number of datapoints in the given dataset
    lenDatapoints = dataset.shape[0]

    # Iterate through each of the new features:
    for newFeature in newFeatures:

        listIsExistAllFeatures = list() # List to store a list of 0s and 1s for each existing feature,
        # if the new feature value exist in the existing features.

        # Iterate through each of the existing feature set and perform the logic to count.
        for existingFeature in existingFeatures:

            listIsExist = list() # List to store '1' if the new feature value exist in the existing feature.

            for value in list(dataset[existingFeature]):

                if str(value) == str(newFeature):

                    listIsExist.append(1)

                else:

```

```

        listIsExist.append(0)

    listIsExistAllFeatures.append(listIsExist)

    arrayCount = np.zeros(lenDatapoints) # Array to store the count of the existing features containing the
    new features.

    # Iterate through each of the list in 'listIsExistAllFeatures' and sum the counts.
    for i in range(0, len(listIsExistAllFeatures)):

        arrayCount = arrayCount + np.array(listIsExistAllFeatures[i])

    dataset[suffix + newFeature] = arrayCount.astype(int)

    return dataset

# Call the encodedFeatures function to generate new features: 'PHY412132', 'PHY337425', 'PHY330576'
data = encodeCatFeatures(data, colPhys, ['PHY412132', 'PHY337425', 'PHY330576'])

# Now remove the original features related to the Physicians
data.drop(columns=['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician'], inplace=True)

#endregion - Physician Features-----

#=====

#region - Claim Diagnosis Features-----

#=====

# Fetch the columns related to the Claims Diagnosis Codes
colDiagCode = [col for col in data.columns if 'ClmDiagnosisCode' in col]

# Call the encodeCatFeatures function to generate new feature for the top 7 Claim Diagnosis Codes
data = encodeCatFeatures(data, colDiagCode, ['4019', '2724', '42731', '25000', '2449', '53081', '4280'],
'ClmDiagCode_')

# For each of the Claim Diagnosis Code Features, replace the values with 1 if there is a value, else replace
with 0 .
for diagCode in colDiagCode:
    data[diagCode] = data[diagCode].apply(lambda x: 1 if not pd.isnull(x) else 0)

#endregion - Claim Diagnosis Features-----

```



```

=====
#
=====

#region - Claim Procedure Features-----

#
=====

# Fetch the columns related to the Claims Procedure Codes
colProcCode = [col for col in data.columns if 'Procedure' in col]

# Call the encodeCatFeatures function to generate new feature for the top 5 Claim Procedure Codes
data = encodeCatFeatures(data, colProcCode, ['9904.0', '8154.0', '66.0', '3893.0', '3995.0'], 'CImProcCode_')

# For each of the Claim Procedure Code Features, replace the values with 1 if there is a value, else replace
with 0 .
for procCode in colProcCode:
    data[procCode] = data[procCode].apply(lambda x: 1 if not pd.isnull(x) else 0)

#endregion - Claim Procedure Features-----

#
=====

#region - Claim Admit Diagnosis Code and Diagnosis Group Code Features-----
---

#
=====

# For each of the Claim Admit Diagnosis Code and Diagnosis Group Code Features,
# replace the values with 1 if there is a value, else replace with 0 .
for code in ['CImAdmitDiagnosisCode', 'DiagnosisGroupCode']:
    data[code] = data[code].apply(lambda x: 1 if not pd.isnull(x) else 0)

#endregion - Claim Admit Diagnosis Code and Diagnosis Group Code Features-----
----

#
=====

```

```

#region - Gender Feature-----
#=====

# Gender Feature has two values: 1 and 2. Replace 2 with 0.
data['Gender'].replace(to_replace=2, value=0, inplace=True)

#endregion - Gender Feature-----
#=====

# Remove some set of Claim Diagnosis/Procedure Code Features
data.drop(columns=['DiagnosisGroupCode', 'ClmProcedureCode_1', 'ClmDiagnosisCode_3',
'ClmDiagnosisCode_4',
                'ClmDiagnosisCode_5', 'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8'],
          inplace=True)

return data

```

4. Handling Data Leakage

- So far whatever features have been generated, do not have any data leakage problem.
- No data imputation has been done based on the mean, standard deviation, median, etc.
- Standardization or Normalization of the data has also not been done so far.
- Unlike the imputation techniques like mean imputation that considers all the data points in the dataset, the new features that have been generated like the physician, diagnosis codes, procedure codes, and other code-related features, have got values by considering the corresponding data point and not all the data points. Hence, this does not lead to any data leakage issue.
- However, for generating new features for the ‘State’ and ‘Country’ features like response encoding and one-hot encoding, we need to consider only the training dataset and not the whole dataset. Otherwise, there will be a data leakage issue.
- We need to ensure that we do response encoding, one-hot encoding, or standardization of the data by only considering the training dataset and not the test dataset while training a model. Otherwise, the Model would have access to the data which it would not have in real-time predictions and it would cause the model to outperform while training and under-perform in real-time.
- We will make use of **pipelines** to overcome the data leakage issues.

5. Encoding Categorical Features: Response Encoding

- We would use Response Encoding for the 'State' and 'Country' features.
- For each categorical feature, Response Encoding creates 'n' number of new features for an 'n' class classification problem.
- As this problem is a case of binary classification to identify a claim record as either fraud or non-fraud, there would be a total of 4 new features created for 'State' and 'Country' features (two for each feature).
- The value in the new feature signifies the **likelihood probability** of the feature given a particular class label.
- The below image explains the concept of Response Encoding:

The below image explains the concept of Response Encoding.

Train Data		
	State	
	A	
	B	
	C	
	A	
	A	
	B	
	A	
	A	
	C	
	C	

Resonse table(only from train)						
	State		Class=0		Class=1	
	A		3		2	
	B		0		2	
	C		1		2	

Encoded Train Data						
	State_0		State_1		class	
	3/5		2/5		0	
	0/2		2/2		1	
	1/3		2/3		1	
	3/5		2/5		0	
	3/5		2/5		1	
	0/2		2/2		1	
	3/5		2/5		0	
	3/5		2/5		1	
	1/3		2/3		1	
	1/3		2/3		0	

Test Data		
	State	
	A	
	C	
	D	
	C	
	B	
	E	

Encoded Test Data				
	State_0		State_1	
	3/5		2/5	
	1/3		2/3	
	1/2		1/2	
	1/3		2/3	
	0/2		2/2	
	1/2		1/2	

- When we fit the response encoding technique on the Train dataset, it generates a Response table and encoded features.

- The test data is transformed based on the response table and new encoded features are generated for the test dataset.
- The response label is built only on the training dataset. For a category that is not there in train data and present in test data, we will encode them with default values. Eg., In our test data if have the State 'D', then we encode it as [0.5, 0.05].

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

class ResponseEncoder(BaseEstimator, TransformerMixin):
    """
    Class to do Response Encoding for the Categorical features.
    This class can be used in the sklearn's Pipeline to avoid data leakage issues
    """
    def __init__(self, categoricalFeatures, className):
        """
        Function to initialize the class members

        Parameter(s):
        -----
        categoricalFeatures: list
            List of features for which the response encoding has to be done to generate new features.
        className: str
            Name of the Class
        """
        self.categoricalFeatures = categoricalFeatures # Categorical Features for which Response Encoding has to
        be done.
        self.responseTable = dict() # Dictionary to store the key:value pair with the 'key' being the categorical
        feature
        # name and its 'value' as the dataframe containing the Response Table.
        self.className = className
        self.classCount = 0 # Number of unique class labels. For binary classification, it will be 2.

    def fit(self, X, y):
        """
        Function called on a Dataset (usually Train Dataset) and Class Label to generate Response Encoded Table.
        This function is called only for the train dataset and not for any cv/test dataset to avoid data leakage.

        Parameters:
        -----
        X: pandas.core.frame.DataFrame
            DataFrame on which the Response Encoding has to be carried out.
        y: pandas.core.series.Series
            Class Labels of the DataFrame
        """
```

Prepare a DataFrame based on the given the dataset and class label, containing just the input features and class labels

```
data = pd.DataFrame()
```

Add the features to the dataframe

```
for col in self.categoricalFeatures:  
    data[col] = X[col]
```

Add the Class Label to the dataframe

```
data['PotentialFraud'] = y
```

Store the count of total unique class labels in the class variable 'classCount'

```
self.classCount = len(y.unique())
```

Iterate through each of the categorical features for which Response Encoding has to be done

```
for feature in self.categoricalFeatures:
```

Dictionary to store the unique categorical feature names and encoded feature name as keys and their values as

list of their corresponding values, class counts and count probabilities

```
dictResponseTable = dict()
```

```
uniqueFeatValues = np.sort(X[feature].unique()) # Array of unique feature values
```

```
uniqueClassLabels = np.sort(y.unique()) # Array of unique class labels
```

Iterate through each of the categorical feature values and generate the Response Table

```
for featureVal in uniqueFeatValues:
```

countClass = list() # List to store the count/frequency of a Class label for a particular feature.

probClass = list() # List to store the probability of occurrence of a class label for a particular feature.

Loop through the unique Class Labels and find the count of the feature value

```
for label in uniqueClassLabels:
```

Append the frequency of Class Label 'label' for the feature 'featureVal' to the list 'countClass'

```
countClass.append(data[(data[feature] == featureVal) & (data[self.className] ==  
label)][self.className].count())
```

Loop through the unique Class Labels and find the likelihood probability of the feature value

```
for label in uniqueClassLabels:
```

Append the likelihood probability of the occurrence of the class 'label' for the feature 'featureVal'

```
probClass.append(countClass[label]/sum(countClass))
```

Prepare a dictionary having keys as features (original and new features) and their values as

```

# feature value (for original features), class counts and class probabilities

# Check if the key already exist or not in the dictionary. If not, create it
if (feature not in dictResponseTable.keys()):
    dictResponseTable[feature] = []
dictResponseTable[feature].append(featureVal) # Append the current iteration's feature value 'feature'

# For each unique class label, add the class label and probability to the corresponding keys in the
dictionary
for label in uniqueClassLabels:

    if (feature + 'Class' + str(label) not in dictResponseTable.keys()):
        dictResponseTable[feature + 'Class' + str(label)] = []
    if (feature + '_' + str(label) not in dictResponseTable.keys()):
        dictResponseTable[feature + '_' + str(label)] = []
    dictResponseTable[feature + 'Class' + str(label)].append(countClass[label])
    dictResponseTable[feature + '_' + str(label)].append(probClass[label])

# Prepare and store the Response Table in the dictionary 'self.responseTable'
self.responseTable[feature] = pd.DataFrame(dictResponseTable)

return self

```

```

def transform(self, X, y= None):
    """
    Function called on a Dataset (Train/Test Dataset) and/or Class Label to generate Response Encoded
    Features.
    This is called to avoid any data leakage. This uses the Response Table already prepared by the fit() method
    and does not consider the test dataset.

```

Parameters:

X: pandas.core.frame.DataFrame

DataFrame on which the Response Encoding has to be done.

y: pandas.core.series.Series

Class Labels of the DataFrame

"""

```

# Get a copy of the input dataframe such the input dataframe is not modified.
xEncoded = X.copy()

```

```

listResponseEncFeat = list() # List to store the names of the response encoded features.

```

```

# Iterate through each of the categorical features for which Response Encoding has to be done

```

```

for feature in self.categoricalFeatures:

```

```

    # Merge with the Response Table Dataframe

```

```

    xEncoded = pd.merge(left=xEncoded, right=self.responseTable[feature], how='left', on=feature)

```

```

# Form the list of the features (original and class count) to be dropped from the Dataset
listResponseEncFeat.extend([col for col in list(self.responseTable[feature].columns) if '_' not in col])

# Fill the values for the datapoints which are not present in the Response Table with equal probabilities of
the class
xEncoded.fillna(1/self.classCount)

# Drop the original features and class count features. Keep only the response encoded features having class
prob.
xEncoded.drop(columns=listResponseEncFeat, inplace=True)

# Convert the values of the dataframe to numeric.
xEncoded.apply(pd.to_numeric)

# Fill the empty/missing values with 0.
xEncoded.fillna(0, inplace=True)

# Return this DataFrame with all the numerical features and the response encoded features for the
categorical features
return xEncoded

```

6. Encoding Categorical Features: One-hot Encoding

- We would do one-hot encoding of the categorical features viz., 'State' and 'Country'.
- To avoid any data leakage issue, we will make use of sklearn's Pipeline.
- To use the Pipeline, we need to create a class with the fit() and transform() method inheriting

BaseEstimator, TransformerMixin class.

```

import pandas as pd
from scipy.sparse import hstack
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.base import BaseEstimator, TransformerMixin

class OneHotEncoder(BaseEstimator, TransformerMixin):
    """
    Class to do One-hot Encoding for the Categorical features.
    This class can be used in the sklearn's Pipeline to avoid data leakage issues
    """
    def __init__(self, categoricalFeatures):
        """
        Function to initialize the class members

        Parameter(s):
        -----
        categoricalFeatures: list
            List of features for which the response encoding has to be done to generate new one-hot encoded
            features.

```

```

'''
    self.categoricalFeatures = categoricalFeatures # Categorical Features for which One-hot Encoding has to
be done.
    self.countVectorizers = dict() # Dictionary of CountVectorizer object for each features to be one-hot
encoded.

    # Add CountVectorizer object for each feature to the dictionary 'self.countVectorizers'
    for feature in categoricalFeatures:

        # 'CountVectorizer not considering single letter text': https://stackoverflow.com/a/63339533/16007029
        self.countVectorizers[feature] = CountVectorizer(tokenizer=lambda x: x.split())

def fit(self, X, y=None):
    '''
    Function called on a Dataset (usually Train Dataset) to generate One-hot Encoded Features.
    This function is called only for the train dataset and not for any cv/test dataset to avoid data leakage.

    Parameters:
    -----
    X: pandas.core.frame.DataFrame
        DataFrame on which the Response Encoding has to be carried out.
    '''

    # Fit the CountVectorizer object for each feature, on the Train Data feature
    for feature in self.categoricalFeatures:

        self.countVectorizers[feature].fit(pd.Series(X[feature]).astype(str).values)

    return self

def transform(self, X, y=None):
    '''
    Function called on a Dataset (Train/Test Dataset) and/or Class Label to generate One-hot Encoded
Features.
    This is called to avoid any data leakage. This uses the CountVectorizer object already prepared by the fit()
method
    and does not consider the test dataset.

    Parameters:
    -----
    X: pandas.core.frame.DataFrame
        DataFrame on which the One-hot Encoding has to be done.
    '''

    # Create a copy of the dataframe so that it does not modify the input dataframe
    xEncoded = X.copy()

```



```
oneHotEncodedFeatures = dict() # Dictionary to store the feature name as key and its one-hot encoded csr matrix in its value
```

```
# Transform each of the categorical feature to convert to one-hot encoding
```

```
for feature in self.categoricalFeatures:
```

```
    oneHotEncodedFeatures[feature] =  
self.countVectorizers[feature].transform(pd.Series(xEncoded[feature]).astype(str).values)
```

```
# Remove the original categorical features from the Dataset
```

```
xEncoded.drop(columns=self.categoricalFeatures, inplace=True)
```

```
# Stack (horizontally) the new one-hot encoded features to the dataset
```

```
for feature in self.categoricalFeatures:
```

```
    xEncoded = hstack((xEncoded, oneHotEncodedFeatures[feature])).tocsr()
```

```
# Return this DataFrame with all the numerical features and the one-hot encoded features for the categorical features
```

```
return xEncoded
```

7. Encoding Numerical Features: Column Standardization

- We will standardize the below numerical features available in the dataset:
 - i. Race
 - ii. ClaimSettlementDelay
 - iii. TreatmentDuration
 - iv. Age
 - v. TotalClaimAmount
 - vi. IPTotalAmount
 - vii. OPTotalAmount
 - viii. UniquePhysCount
 - viii. PhysRoleCount
- Again for standardizing the features, we need to ensure that there is no data leakage.
- Hence, we will create a class with fit() and transform() method inheriting BaseEstimator, TransformerMixin class.

```
from sklearn.preprocessing import StandardScaler  
from sklearn.base import BaseEstimator, TransformerMixin
```

```
class Standardize(BaseEstimator, TransformerMixin):
```

```

"""
Class to do standardization of the numerical features.
This class can be used in the sklearn's Pipeline to avoid data leakdage issues
"""
def __init__(self, numericalFeatures):
    """
    Function to initialize the class members

    Parameter(s):
    -----
    numericalFeatures: list
        List of numerical features to be standardized.
    """
    self.numericalFeatures = numericalFeatures # Numerical Features to be standardized.
    self.standardScaler = StandardScaler() # Object of StandardScaler.

def fit(self, X, y=None):
    """
    Function called on a Dataset (usually Train Dataset) to fit the train dataset.
    This function is called only for the train dataset and not for any cv/test dataset to avoid data leakage.

    Parameters:
    -----
    X: pandas.core.frame.DataFrame
        Dataset to be considered for standardization.
    """

    # Fit the StandardScaler object only on the numerical features.
    self.standardScaler.fit(X[self.numericalFeatures])

    return self

def transform(self, X, y=None):
    """
    Function called on a Dataset (Train/Test Dataset) to standardize the data based on the Train Dataset.
    This is called to avoid any data leakage. This uses the standardScaler object already prepared by the fit()
method
and does not consider the test dataset.

    Parameters:
    -----
    X: pandas.core.frame.DataFrame
        Dataset to be standardized.
    """

    # Create a copy of the dataframe so that it does not modify the input dataframe
    xStandardized = X.copy()

```

```
# Standardize the numerical features of the dataset.  
xStandardized[self.numericalFeatures] =  
self.standardScaler.transform(xStandardized[self.numericalFeatures])  
  
# Return the standardized dataset  
return xStandardized
```

Machine Learning Models — Hyperparameter Tuning and Model Training

- Here, we will do hyperparameter tuning on various Machine Learning Models to find the best hyperparameters.
- With the best hyperparameters, we will train the models with different data sampling techniques and do the evaluation.

Split the Dataset for Training and Testing

- We would split the data into a ratio of **80:20**. 80% of the data will be used for doing k-fold cross-validation hyperparameter tuning of the model and the rest 20% of the data (unseen) will be used in evaluating the model.

Sampling — Oversampling and Undersampling

- We will use different sampling techniques for sampling the data before training the Models.
- We would do 4 kinds of data sampling:
 - 1. Raw Data:** Use raw data without any undersampling or oversampling.
 - 2. Balanced Class Weights:** Use balanced class weights.
 - 3. Random Undersampling:** Randomly undersample the majority class data to have an equal number of data points as that in the minority class data.
 - 4. Oversampling using SMOTE:** Oversample the minority class data to have an equal number of data points as that in the majority class data, using **SMOTE (Synthetic Minority Over-sampling Technique)**.
- We would use the below Machine Learning Models for doing hyperparameter tuning and evaluation, with each of the 4 sampling techniques:
 1. Logistic Regression.
 2. Decision Tree.
 3. Random Forest.
 4. XGBoost.
 5. LightGBM.

- A sample code for doing hyperparameter tuning, training the model with the best hyperparameters

found, and doing the evaluation is shown below:

```
%%time
# Define an object of XGBoost Classifier Model
xgbClf4 = XGBClassifier()

# Define different values for the Hyperparameters to do tuning
params = {
    'learning_rate': [0.01, 0.03, 0.05, 0.1, 0.15, 0.2],
    'n_estimators': [50, 100, 200, 500, 1000, 2000],
    'max_depth': [3, 5, 10],
    'colsample_bytree': [0.1, 0.3, 0.5, 1],
    'subsample': [0.1, 0.3, 0.5, 1]
}

# Create Pipeline:
# 1. Standardization of numerical features.
# 2. Response Encoding of 'State' and 'Country' features.
# 3. RandomizedSearchCV using XGBoost Classifier
# Note: The order of Standardization and Response Encoding does not matter here because we are doing these
encodings on different features.
pipelineXGB4 = sklearnPipeline(steps=[
    ('standardization', Standardize(numericalFeatures=featuresToStd)),
    ('responseEncoding', ResponseEncoder(categoricalFeatures=['State', 'Country'],
className='PotentialFraud')),
    ('randomizedSearchCV', RandomizedSearchCV(xgbClf4, param_distributions=params, n_iter=5, cv=3,
verbose=10, n_jobs=-1, random_state=1))
])

# Fit the Pipeline on the Train Data with SMOTE Oversampled data
pipelineXGB4.fit(xTrainSMOTE, yTrainSMOTE)
```

Fitting 3 folds for each of 5 candidates, totalling 15 fits
 CPU times: user 51min 37s, sys: 3.71 s, total: 51min 40s
 Wall time: 1h 32min 36s

```
Out[78]: Pipeline(steps=[('standardization',
                           Standardize(numericalFeatures=['Race', 'ClaimSettlementDelay',
                                                           'TreatmentDuration', 'Age',
                                                           'TotalClaimAmount',
                                                           'IPTotalAmount',
                                                           'OPTotalAmount',
                                                           'UniquePhysCount',
                                                           'PhysRoleCount'])),
                          ('responseEncoding',
                           ResponseEncoder(categoricalFeatures=['State', 'Country'],
                                             className='PotentialFraud')),
                          ('randomizedSearchCV',
                           RandomizedSea...

                           reg_lambda=None,
                           scale_pos_weight=None,
                           subsample=None,
                           tree_method=None,
                           validate_parameters=None,
                           verbosity=None),

                           n_iter=5, n_jobs=-1,
                           param_distributions={'colsample_bytree': [0.1,
                                                                      0.3,
                                                                      0.5,
                                                                      1],
                                                'learning_rate': [0.01,
                                                                0.03,
                                                                0.05,
                                                                0.1,
                                                                0.15,
                                                                0.2],
                                                'max_depth': [3, 5,
                                                            10],
                                                'n_estimators': [50,
                                                                100,
                                                                200,
                                                                500,
                                                                1000,
                                                                2000],
                                                'subsample': [0.1, 0.3,
                                                            0.5,
                                                            1]}],

                           random_state=1, verbose=10))])
```

```
In [79]: 1 print('Best Parameters found:', pipelineXGB4.steps[2][1].best_params_)

Best Parameters found: {'subsample': 1, 'n_estimators': 1000, 'max_depth': 10, 'learning_rate': 0.1, 'colsample_bytree': 0.
```

- After getting the best hyperparameters using k-fold cross-validation and random search, train the model using the best hyperparameters as shown below:

```
%%time
# Create Pipeline:
# 1. Standardization of numerical features.
# 2. Response Encoding of 'State' and 'Country' features.
# 3. RandomizedSearchCV using XGBoost Classifier
pipelineXGB4 = sklearnPipeline(steps=[
    ('standardization', Standardize(numericalFeatures=featuresToStd)),
    ('responseEncoding', ResponseEncoder(categoricalFeatures=['State', 'Country'],
                                         className='PotentialFraud')),
    ('xgboost', XGBClassifier(subsample=1, n_estimators=1000, max_depth=10, learning_rate=0.1,
                              colsample_bytree=0.3))
])
```

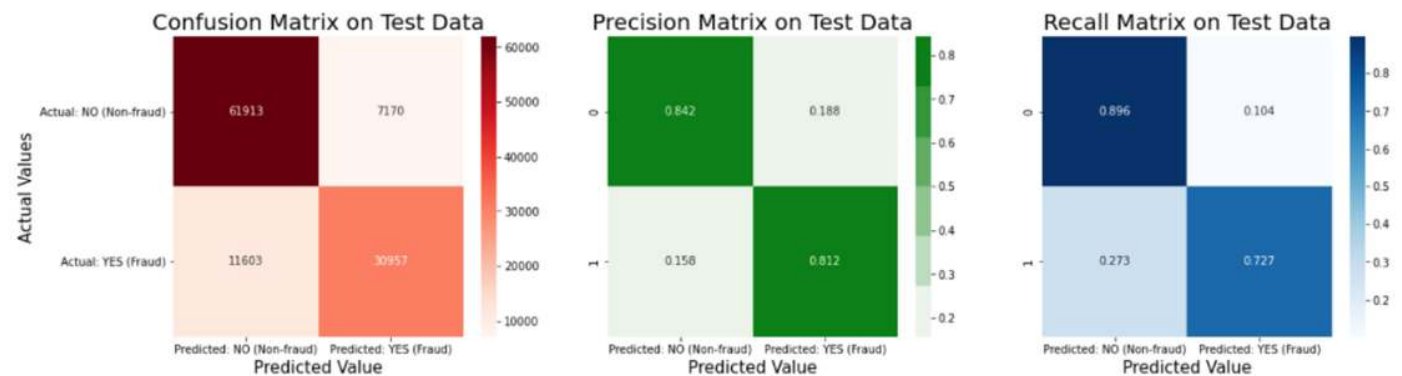
Fit the Pipeline on the Train Data with SMOTE Oversampling

pipelineXGB4.fit(xTrainSMOTE, yTrainSMOTE)

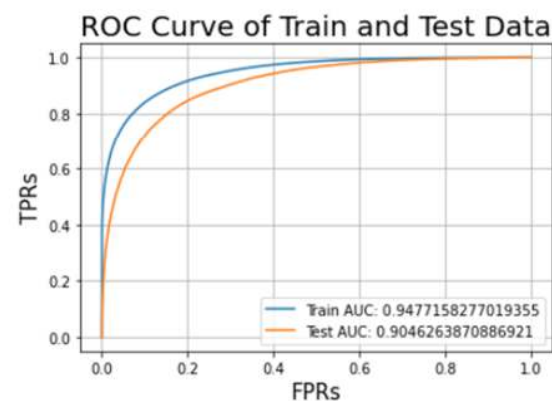
- Evaluate the trained model on various performance metrics discussed before. The result looks like this:

Press enter or click to view image in full size

Log-loss of the Model on Train Dataset: 0.3150195194700413
Log-loss of the Model on Test Dataset: 0.3935665350508559
F1-Score of the Model on Test Data: 0.7673355063393111



Percentage of misclassified points (Test Data): 16.815205610741383 %
Sum of columns in the Precision Matrix: [1. 1.]
Sum of rows in the Recall Matrix: [1. 1.]
False Positive Rate (FPR) on Test Data: 0.10378819680673972
False Negative Rate (FNR) on Test Data: 0.27262687969924815
Balanced Accuracy Score (BACC) on Test Data: 0.811792461747006
Matthew's Correlation Coefficient (MCC) on Test Data: 0.638667415195444



Comparison of Models

- We did the hyperparameter tuning on various models using different sampling techniques, trained the models, and evaluated them.
- The table below shows a comparison between various models used with different sampling techniques.
- The green highlighted cells indicate the best value of the metric across all the models.

Model	Sampling Technique	Encoding Technique	Train Data		Test Data							
			AUC (Train Data)	Log-loss (Train Data)	Log-loss (Test Data)	AUC (Test Data)	F1-Score	Percentage Misclassification	FPR	FNR	BACC	MCC
Logistic Regression	None	One-hot Encoding*	0.74097	0.57481	0.57316	0.74464	0.54777	29.59	0.15188	0.52982	0.65915	0.34693
Logistic Regression	Balanced Class Weights	One-hot Encoding*	0.74139	0.59742	0.5965	0.74497	0.62397	31.29	0.30928	0.31889	0.68591	0.36293
Logistic Regression	Random Undersampling	One-hot Encoding*	0.74152	0.5988	0.59831	0.74484	0.62432	31.49	0.3158	0.31353	0.68533	0.36138
Logistic Regression	SMOTE	One-hot Encoding*	0.79221	0.54082	0.60104	0.71757	0.57263	31.72	0.23993	0.44257	0.65875	0.32107
Decision Tree	None	Response Encoding**	0.86694	0.43362	0.62828	0.83325	0.68552	22.56	0.14606	0.35484	0.74955	0.51308
Decision Tree	Balanced Class Weights	Response Encoding**	0.8656	0.45284	0.63311	0.83151	0.70404	24.1	0.23682	0.24791	0.75764	0.50504
Decision Tree	Random Undersampling	Response Encoding**	0.86093	0.45972	0.64142	0.82885	0.6991	24.21	0.22995	0.26201	0.75401	0.49935
Decision Tree	SMOTE	Response Encoding**	0.89373	0.40121	0.48393	0.85139	0.70651	21.93	0.16501	0.30749	0.76374	0.53181
Random Forest	None	Response Encoding**	0.89265	0.40513	0.46371	0.84857	0.70006	21.47	0.13582	0.34274	0.76072	0.53673
Random Forest (with Calibration)	None	Response Encoding**	0.88661	0.42109	0.46689	0.84814	0.69911	21.55	0.13685	0.3432	0.75997	0.535
Random Forest	Balanced Class Weights	Response Encoding**	0.97359	0.27579	0.48186	0.84307	0.7075	22.39	0.18353	0.28954	0.76346	0.52609
Random Forest	Random Undersampling	Response Encoding**	0.97355	0.28446	0.49672	0.84211	0.71269	23.27	0.22632	0.24297	0.76535	0.52084
Random Forest	SMOTE	Response Encoding**	0.94208	0.32997	0.39447	0.90339	0.76219	17.6	0.12424	0.26005	0.80785	0.62339
XGBoost	None	Response Encoding**	0.92598	0.35781	0.46636	0.84666	0.69653	21.75	0.13882	0.34523	0.75798	0.53072
XGBoost	Random Undersampling	Response Encoding**	0.93511	0.35703	0.48961	0.84457	0.71539	22.9	0.21905	0.24508	0.76793	0.52674
XGBoost	SMOTE	Response Encoding**	0.94772	0.31501	0.39356	0.90463	0.76733	16.81	0.10379	0.27263	0.81179	0.63867
LightGBM	None	Response Encoding**	0.85185	0.46375	0.48479	0.8331	0.6767	22.62	0.13201	0.37904	0.74447	0.50975
LightGBM	Random Undersampling	Response Encoding**	0.85211	0.48259	0.50866	0.82961	0.70215	23.83	0.22301	0.26316	0.75692	0.50582
LightGBM	SMOTE	Response Encoding**	0.89162	0.41068	0.48483	0.8351	0.68415	22.55	0.14312	0.35928	0.7488	0.51288

One-Hot Encoding*: One-hot Encoding of ‘State’ and ‘Country’ features.

Response Encoding**: Response Encoding of ‘State’ and ‘Country’ features.

- We can observe that:
 1. Decision Tree with balanced class weights received the least ‘FNR’ score of 0.24791.
 2. Random Forest with balanced class weights received the minimum ‘Train AUC’ and minimum ‘Train Log-loss’.
 3. XGBoost with SMOTE Oversampling received the least ‘Test Log-loss’, ‘percentage misclassification’, ‘FPR’ and the maximum ‘Test AUC’, ‘F1-Score’, ‘BACC’, and ‘MCC’ score.
- **Conclusion:** As the XGBoost Model with SMOTE oversampling gives the best scores for most of the performance metrics, we will choose **XGBoost with SMOTE Oversampling** as the final model.
- However, the decision to choose the best model depending on which all performance metrics to be considered is subjective to the business need.

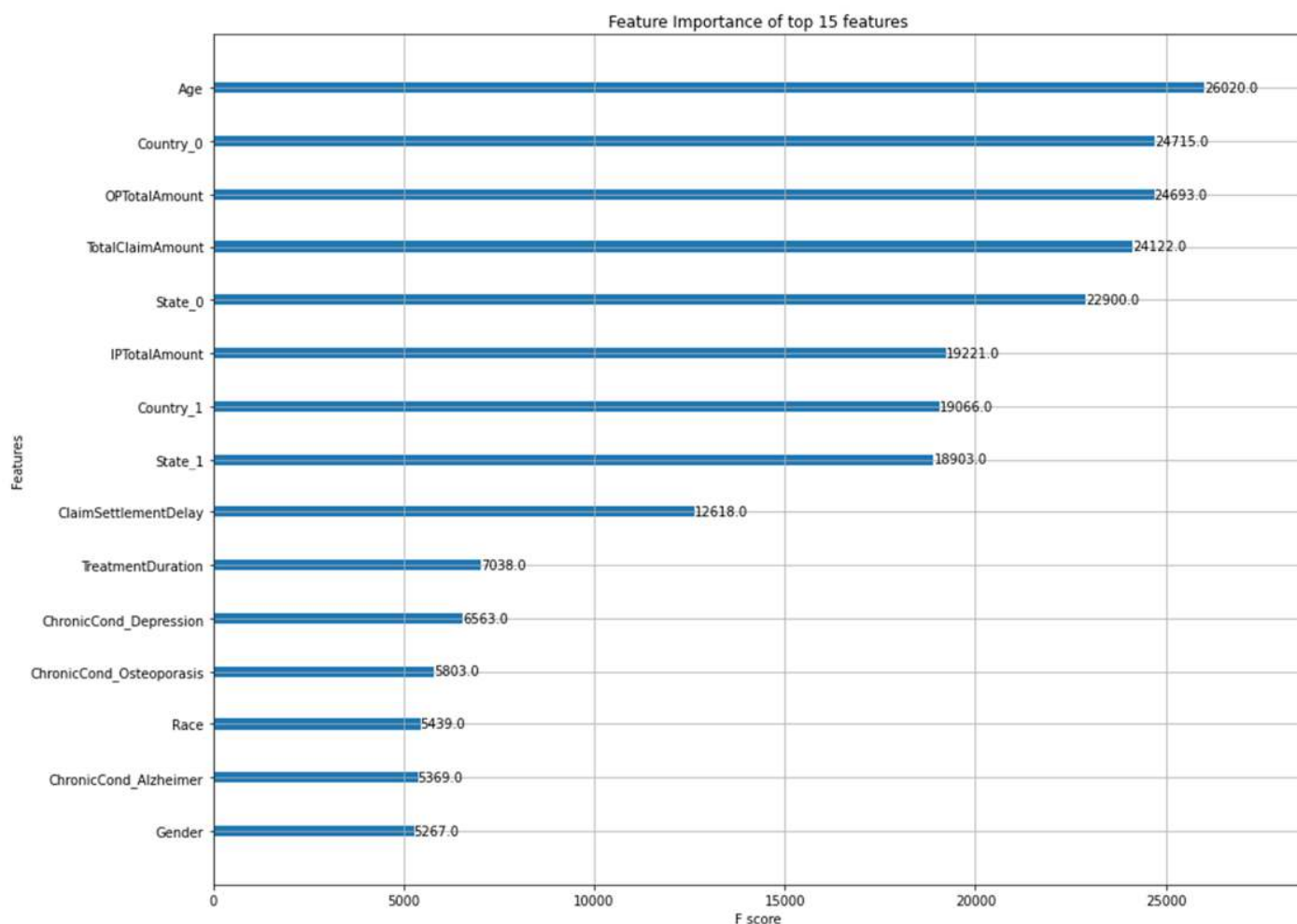
Model Interpretability

Interpretability is very important as the model needs to justify why it classifies a Provider’s claim record as fraudulent or not.

We have tried providing Model Interpretability using approaches:

1. **Global Structure of the Data:** Provides the model interpretability by using the feature importance learned by the model on the whole training dataset.

The figure below shows the model interpretability of XGBoost using the feature importance of the top 15 features:



The interpretability using feature importance remains the same for all query points.

2. Local Structure of the Query Point: Provides the model interpretability by looking at the local structure of the data points.

The figure below shows the model interpretability for a query point using the LIME (Local Interpretable Model Agnostic) technique:



The interpretability using LIME is different for every query point as it considers the local structure of the query points in its neighborhood.

Future Work

The future scope for this problem could be:

- Implementing more complex ensemble modeling techniques like stacking and cascading.
- Develop a deep Multi-layer Perceptron (MLP) Model (Deep Learning Model).
- Explain the Model's interpretability using more advanced techniques like SHAP (SHapley Additive exPlanations).

References

1. [Medicare Fraud Detection using Machine Learning Models.](#)
2. [The Detection of Medicare Fraud using ML Methods with excluded Provider Labels.](#)
3. [A Comparative Study of Using Various Machine Learning and Deep Learning-Based Fraud Detection Models For Universal Health Coverage Schemes.](#)
4. [Credit card fraud detection using machine learning techniques: A comparative analysis.](#)
5. [Medical Provider Fraud Detection — Kaggle](#)
6. [Applied Roots.](#)